

---

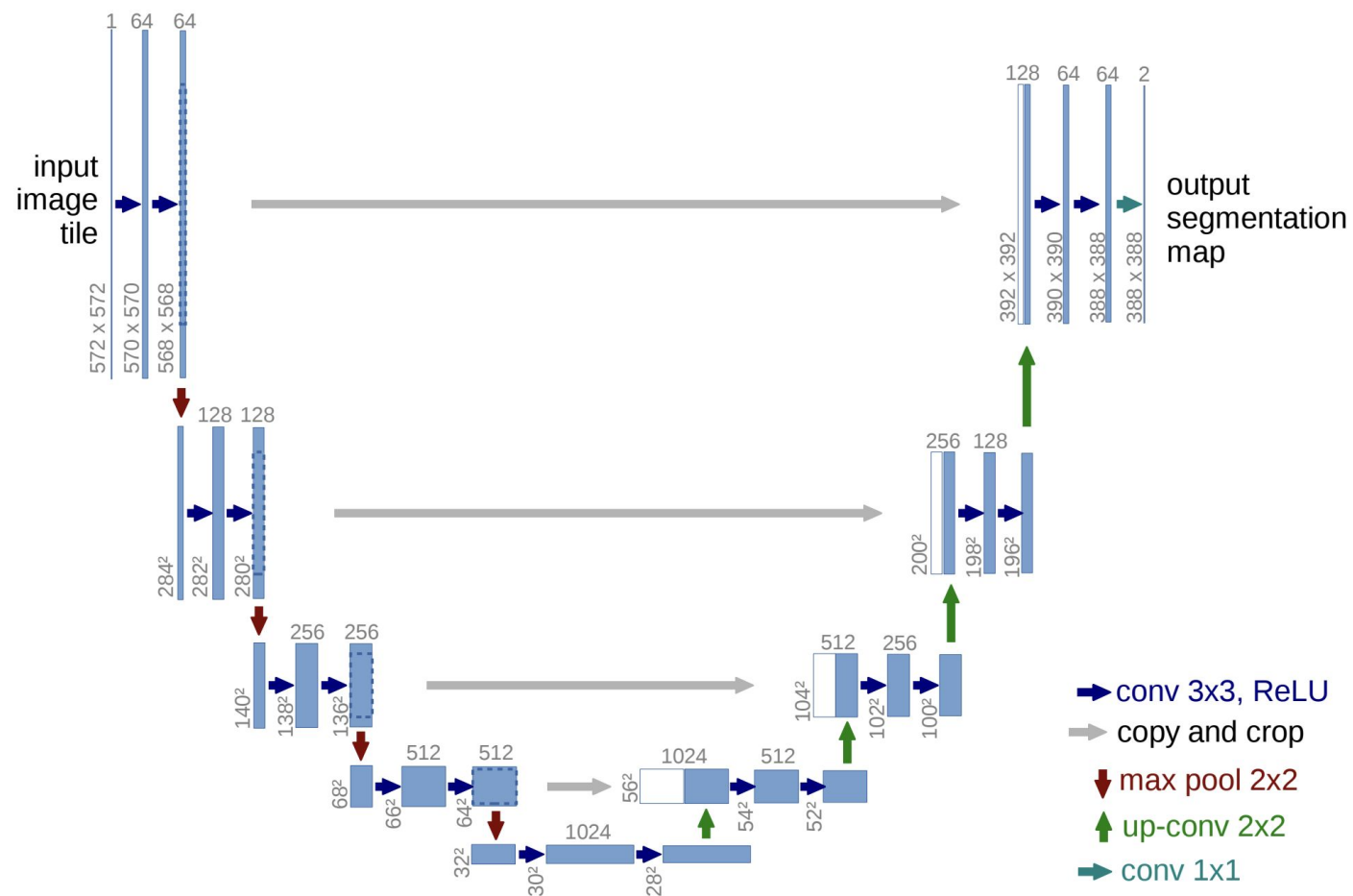
# U-Net

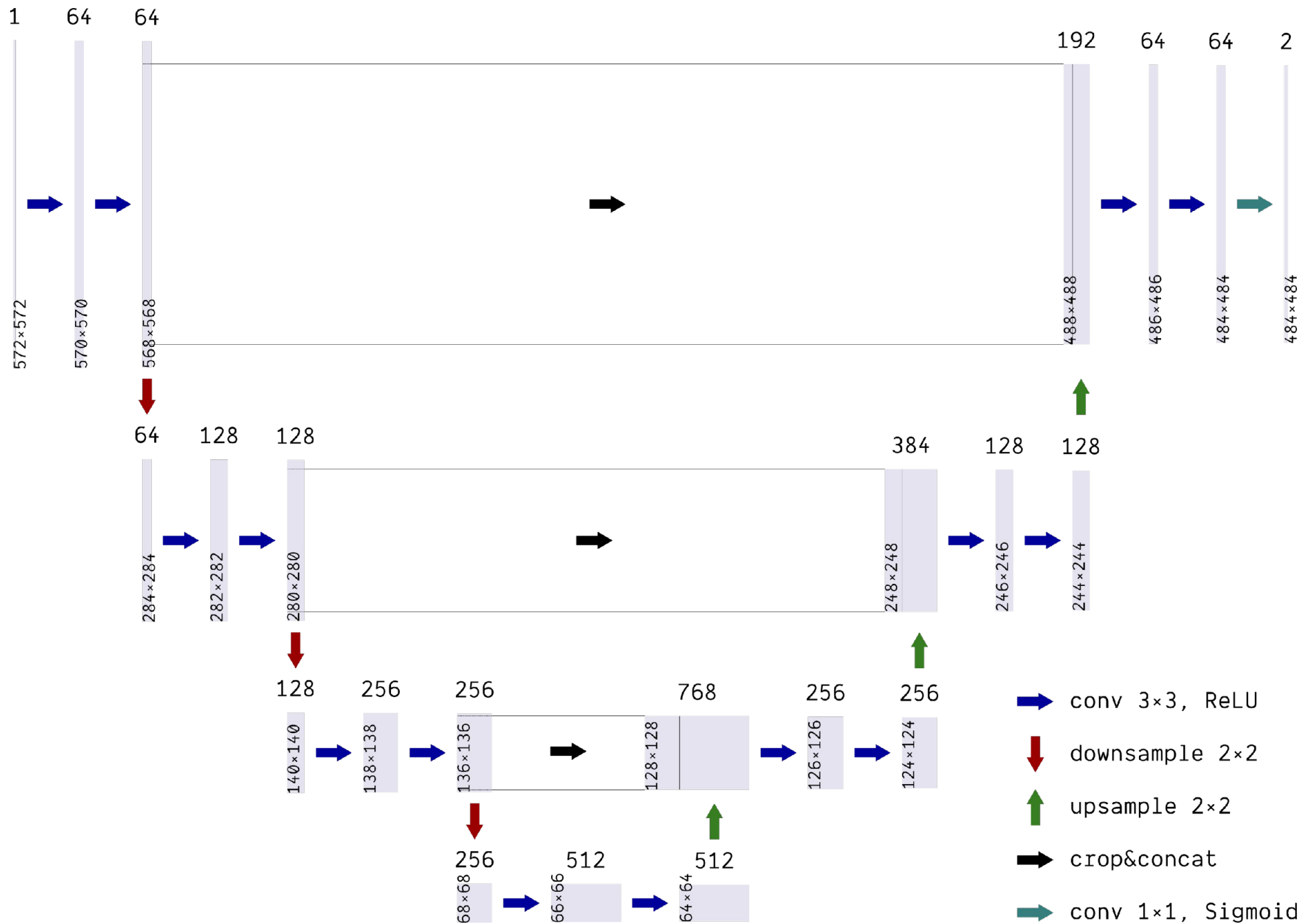
Larissa Heinrich, HHMI Janelia

DL@Janelia 2026

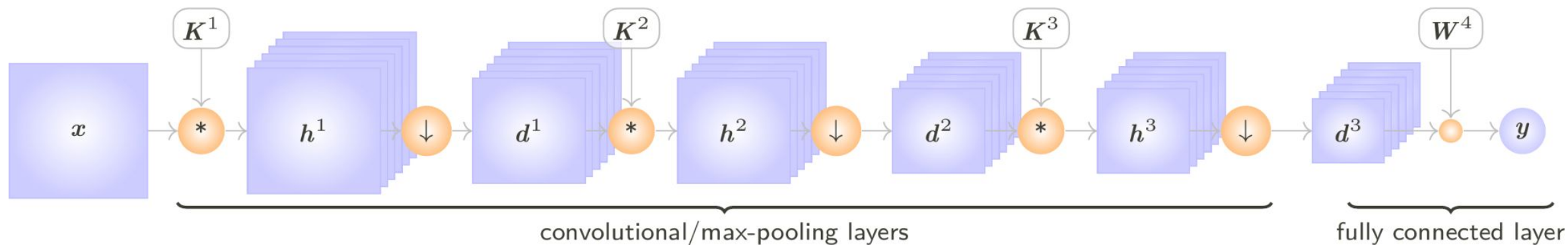
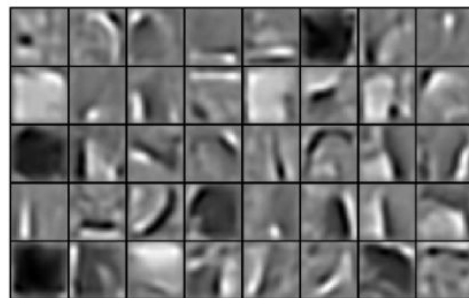
---

# does this look familiar?

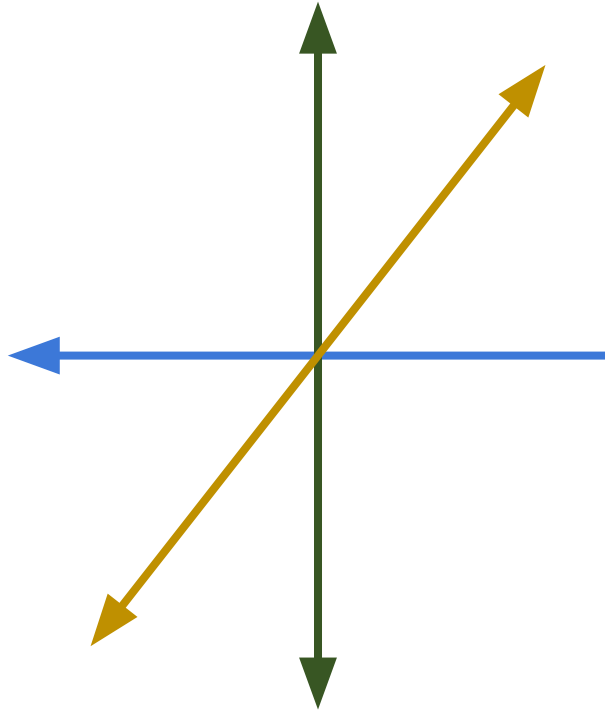




# remember yesterday?



# images



How many  
dimensions  
does this  
image have?

2

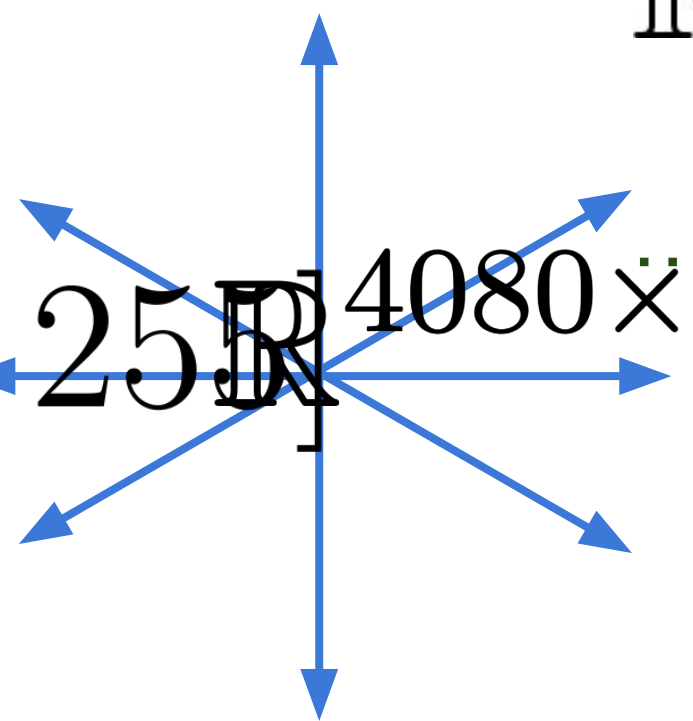
37,601,280



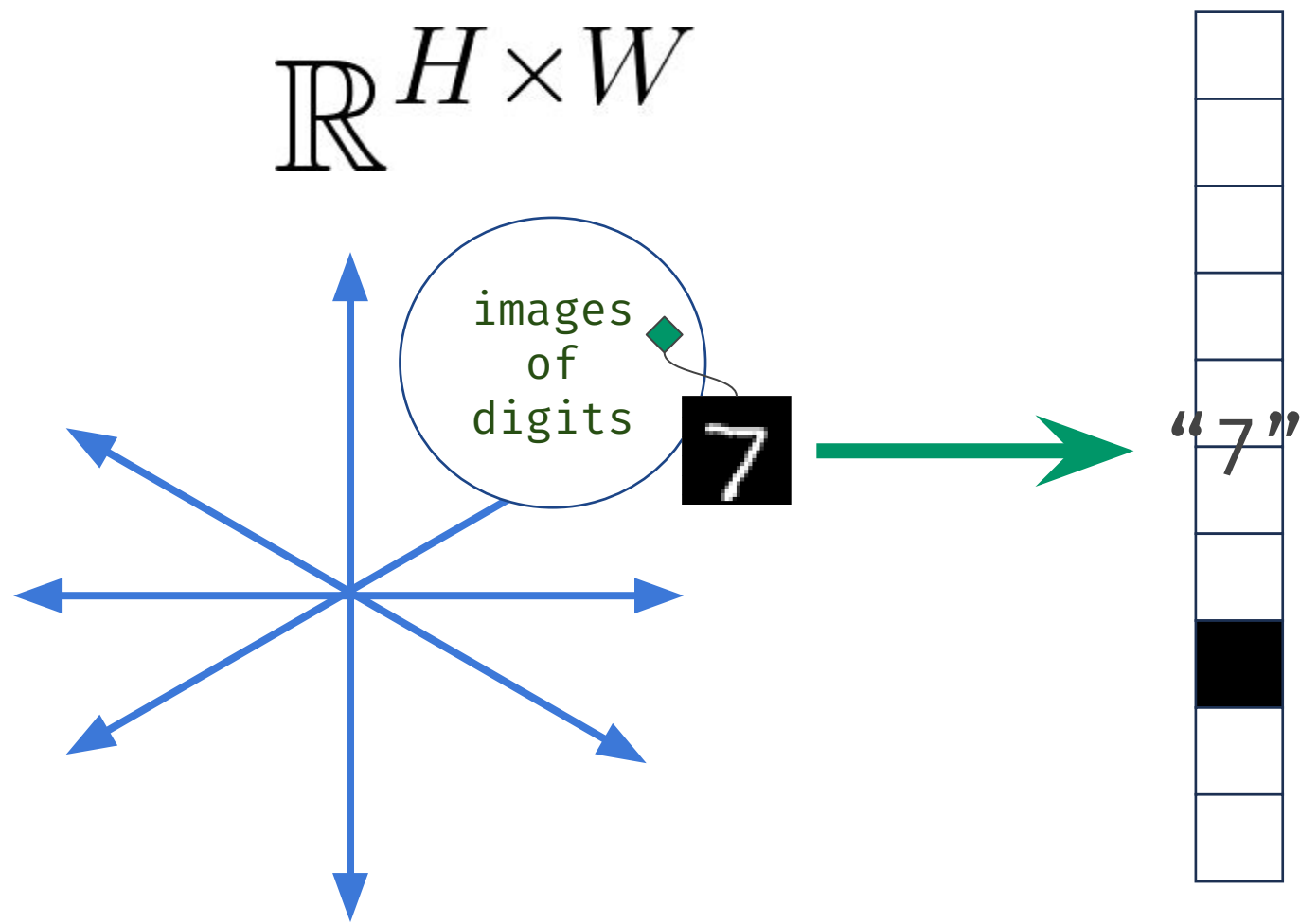
**images**

$\mathbb{R}^{H \times W}$

$[0; 255] \mathbb{R}^{4080 \times 3072 \times 3}$

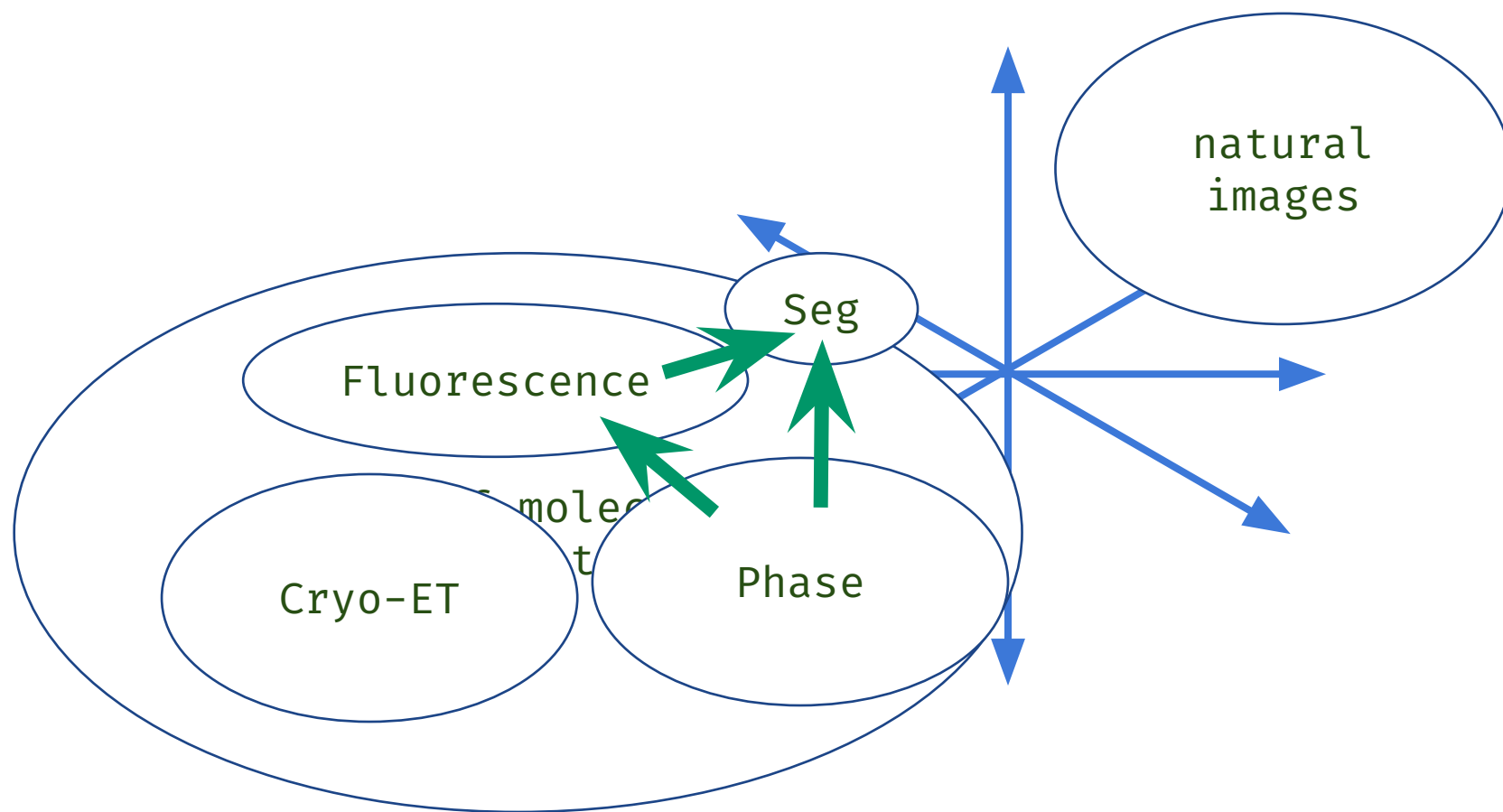
A diagram consisting of a central point from which eight blue arrows radiate outwards in various directions, representing the multidimensional nature of image data.

# image classification



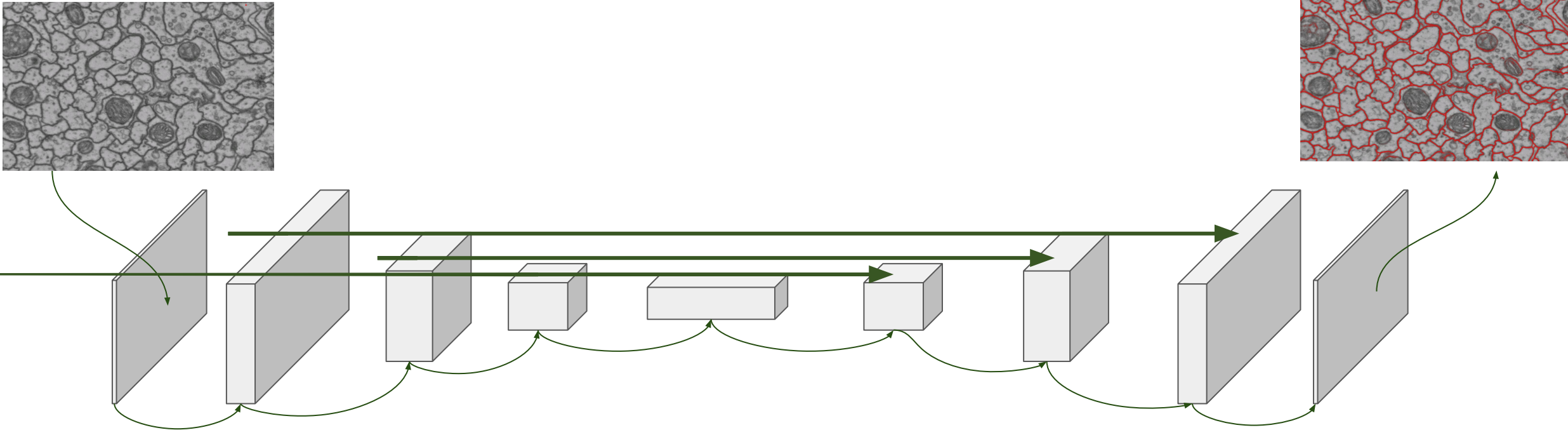
# image-to-image transforms

$$\mathbb{R}^{H \times W}$$

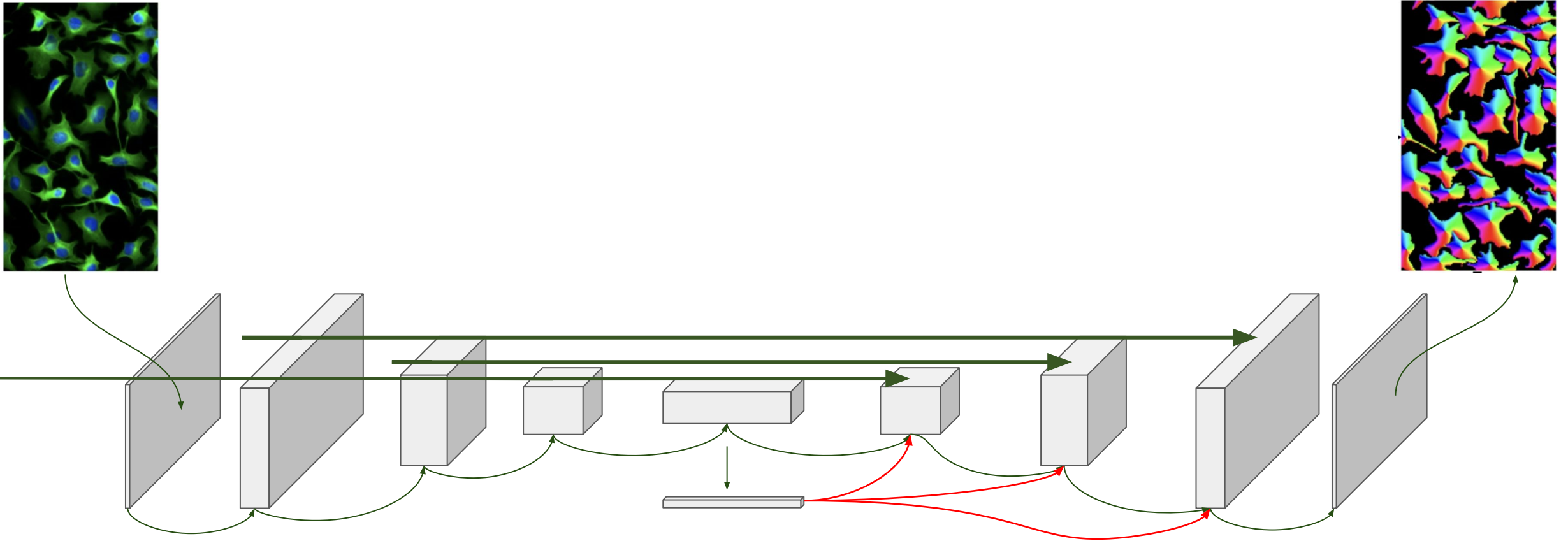




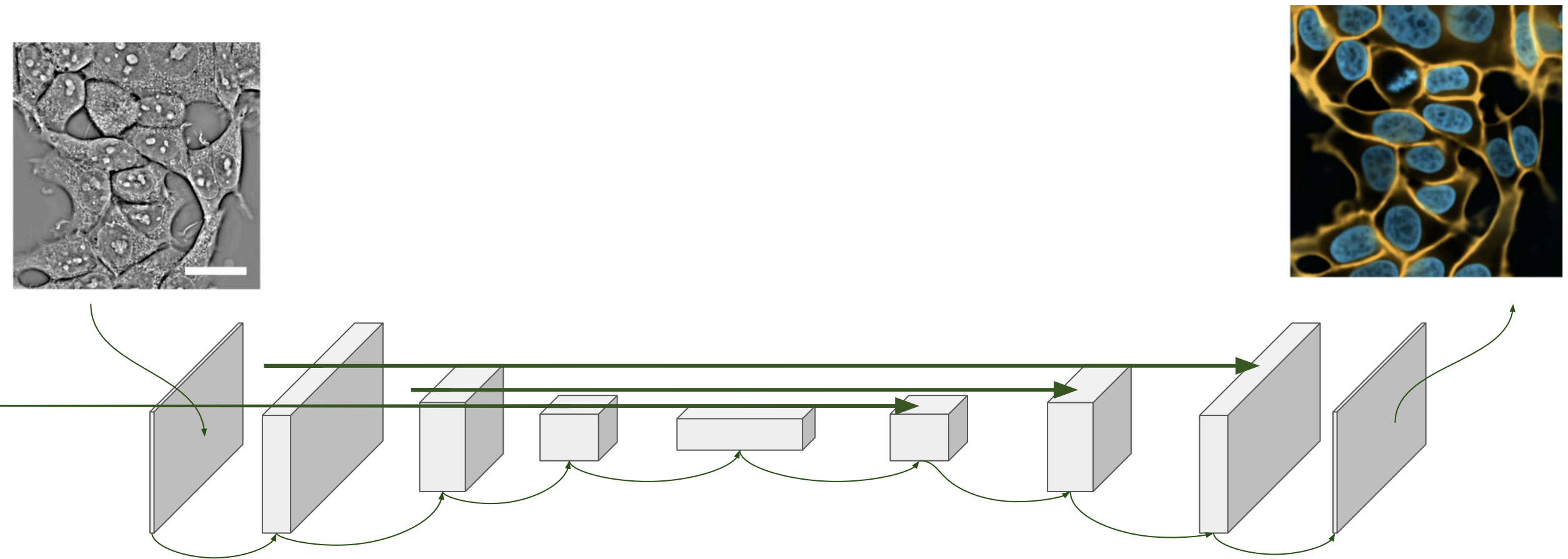
# U-Net in bioimage analysis



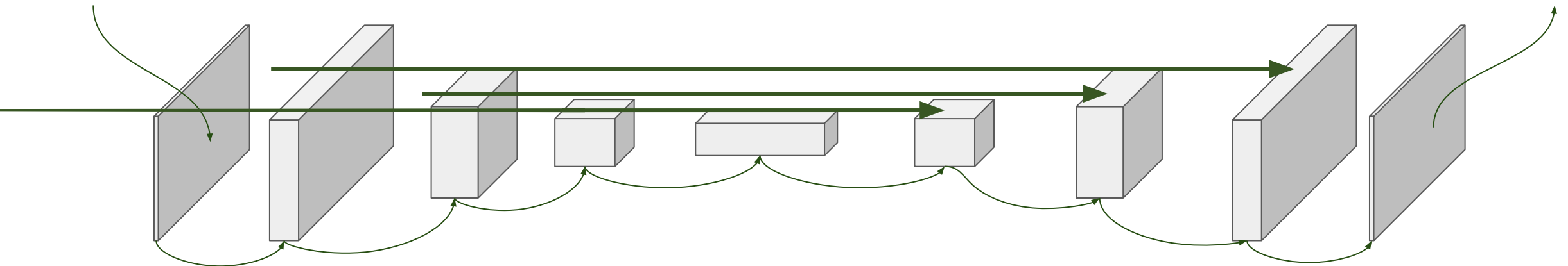
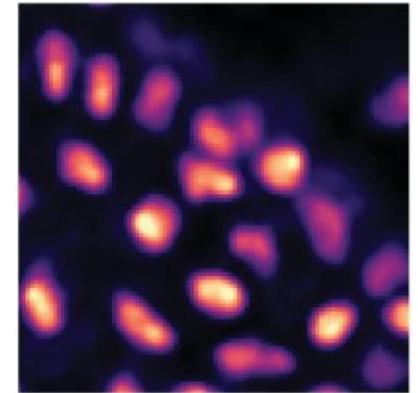
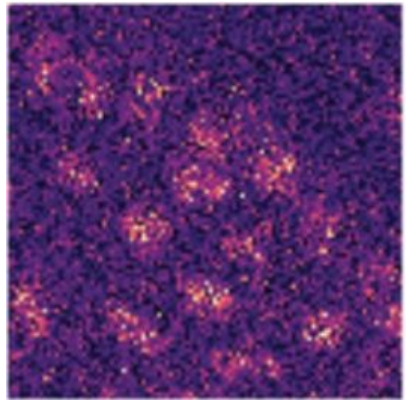
# U-Net in bioimage analysis

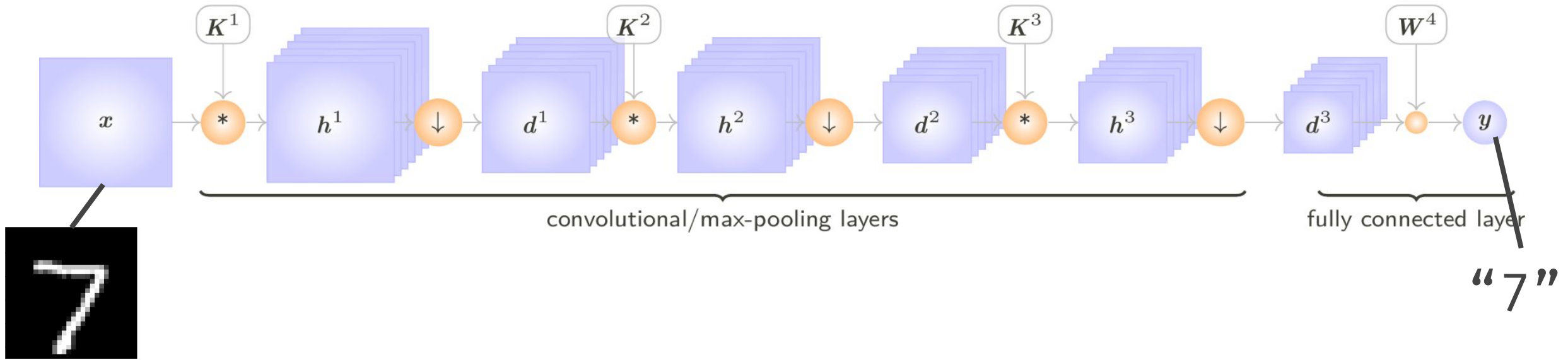


# U-Net in bioimage analysis



# U-Net in bioimage analysis

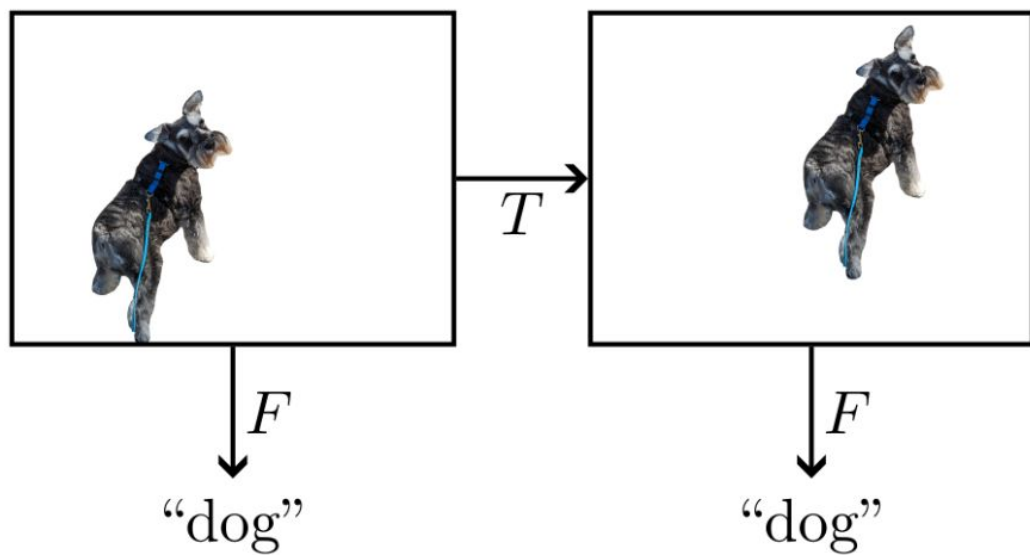




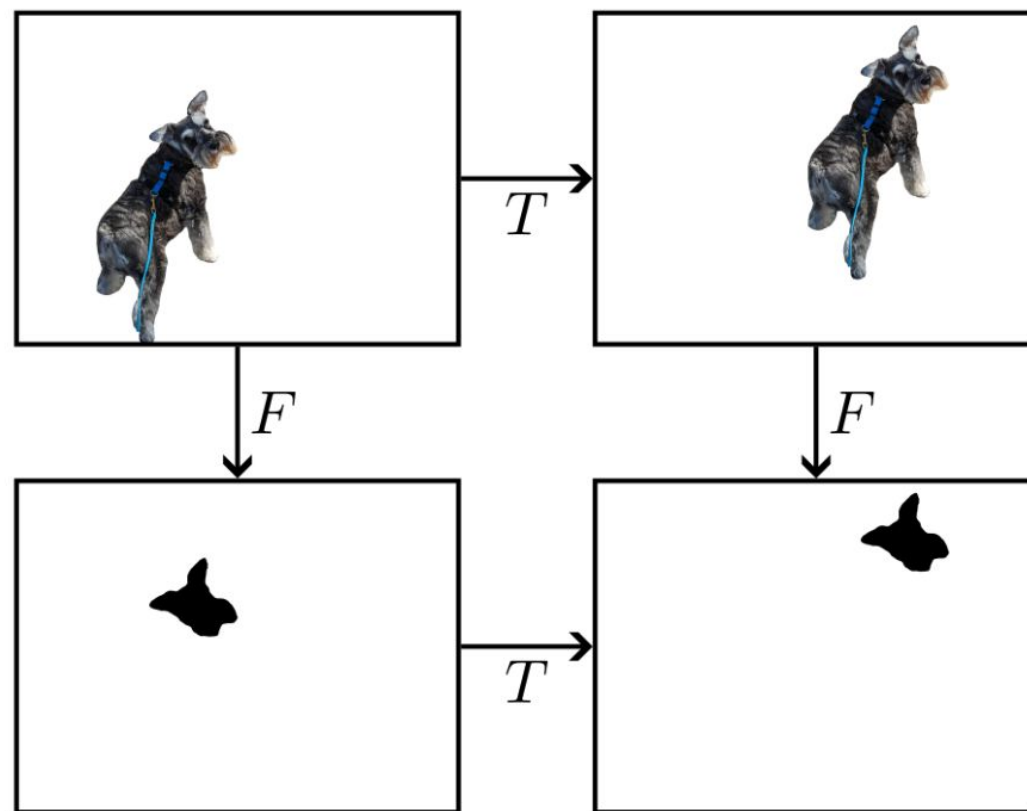
**image-to-image: what's missing?**

# shift equivariance

## Invariance

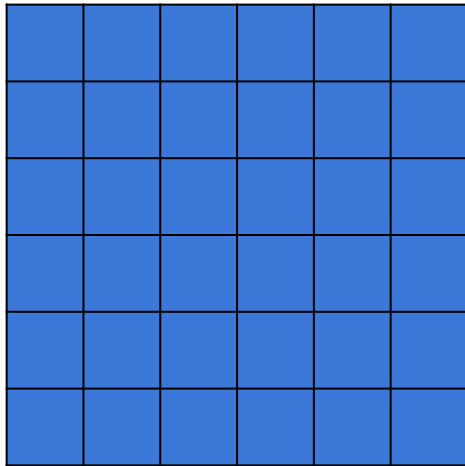


## Equivariance



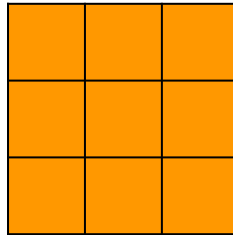


# convolutional layer

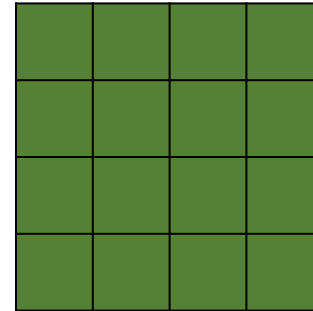


input image

\*

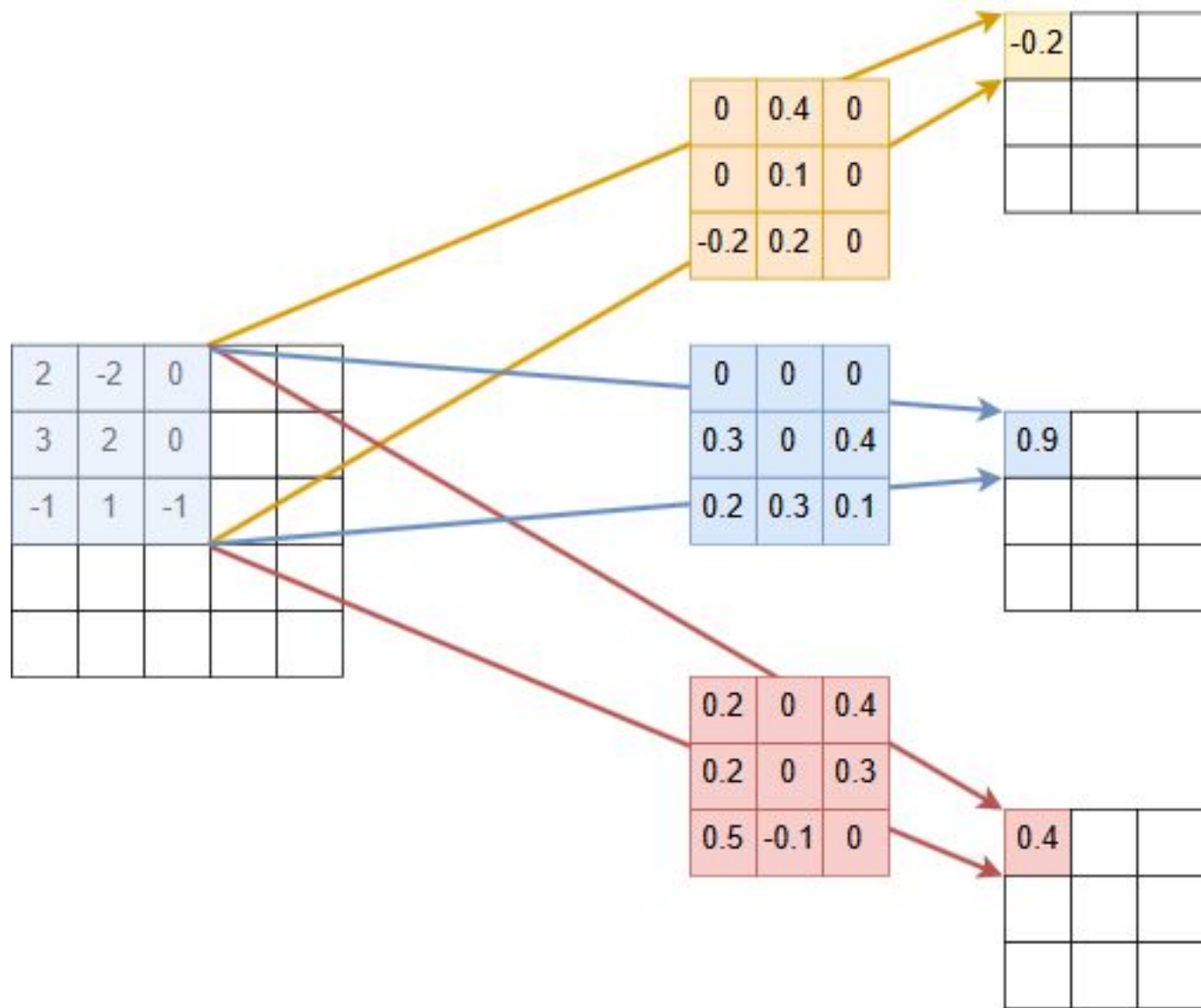


weights



feature map

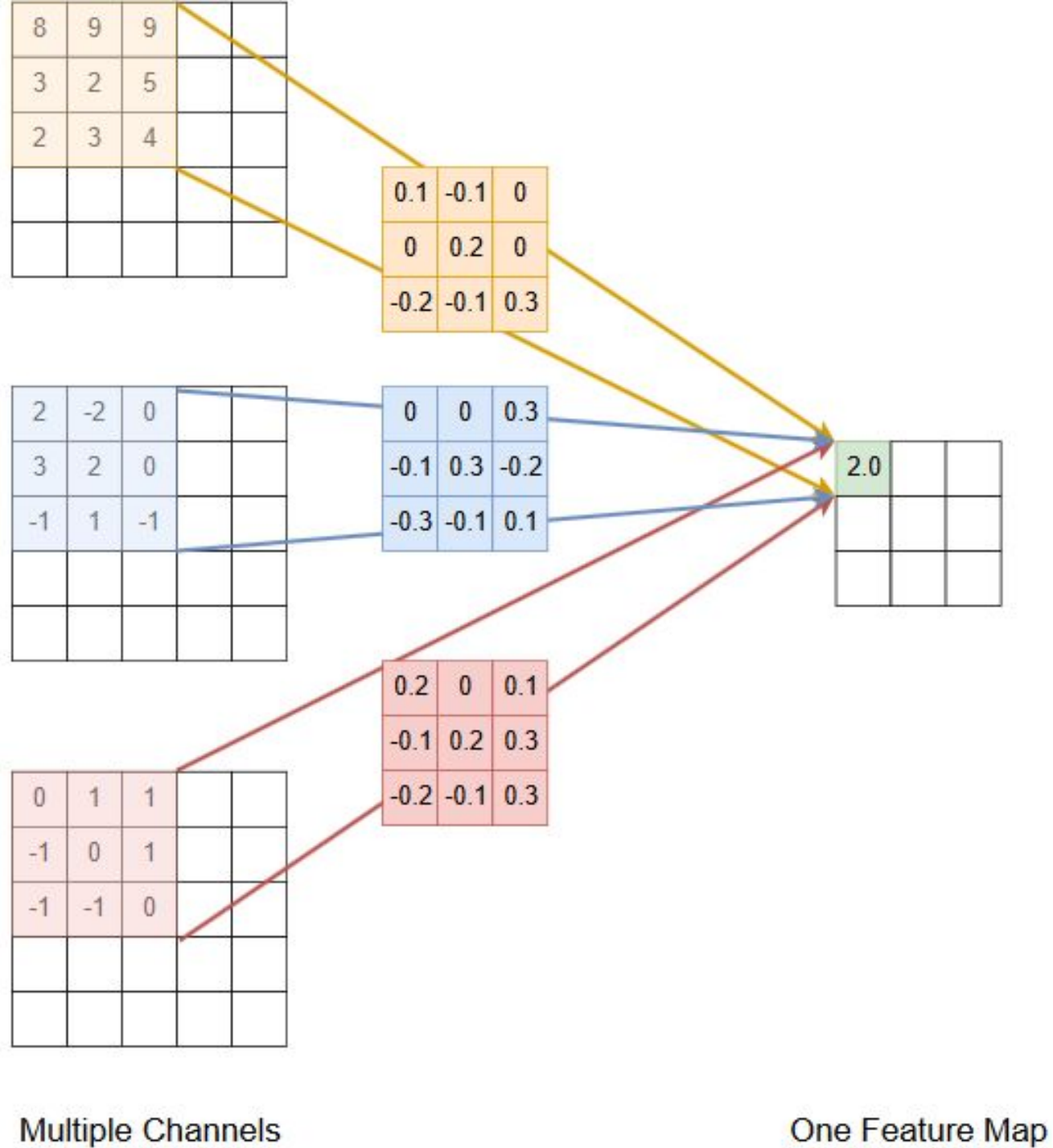
**multi-channel  
output**



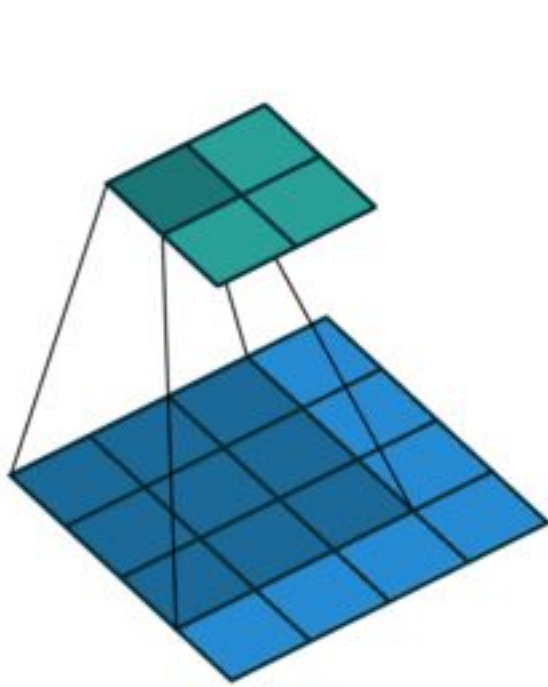
Single Channel

Multiple Feature Maps

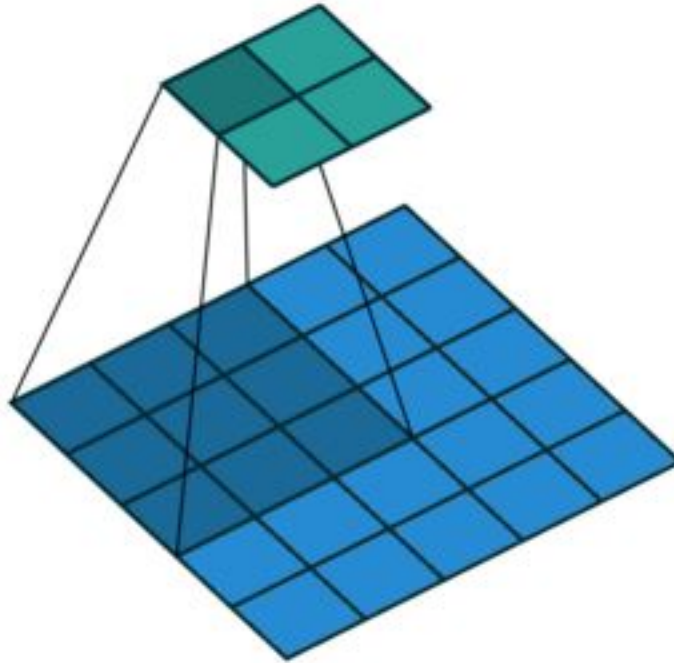
multi-channel  
input



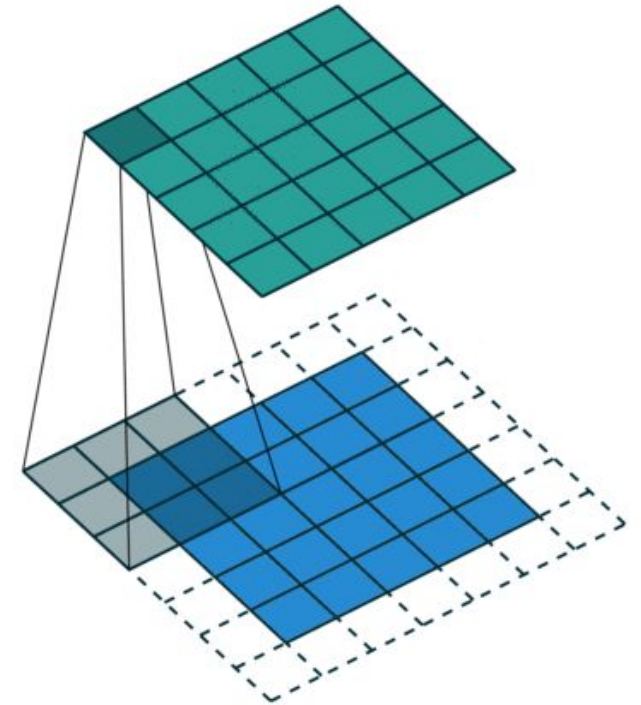
# hyperparameters of convolution



padding = 0,  
stride = 1



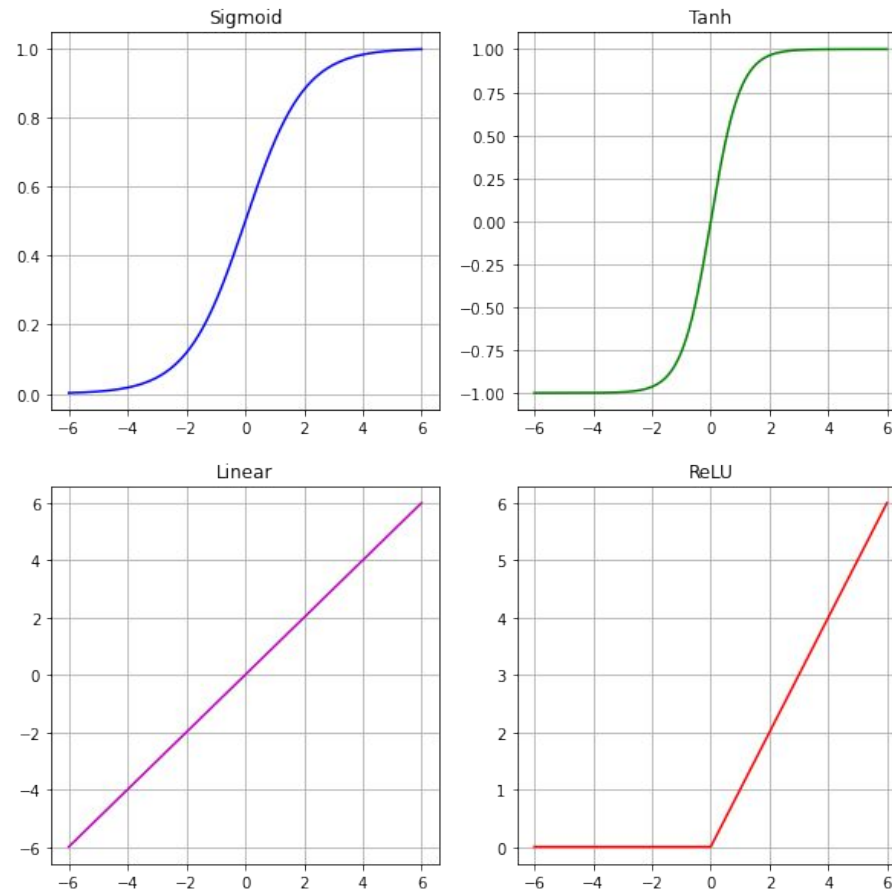
padding = 0,  
stride = 2



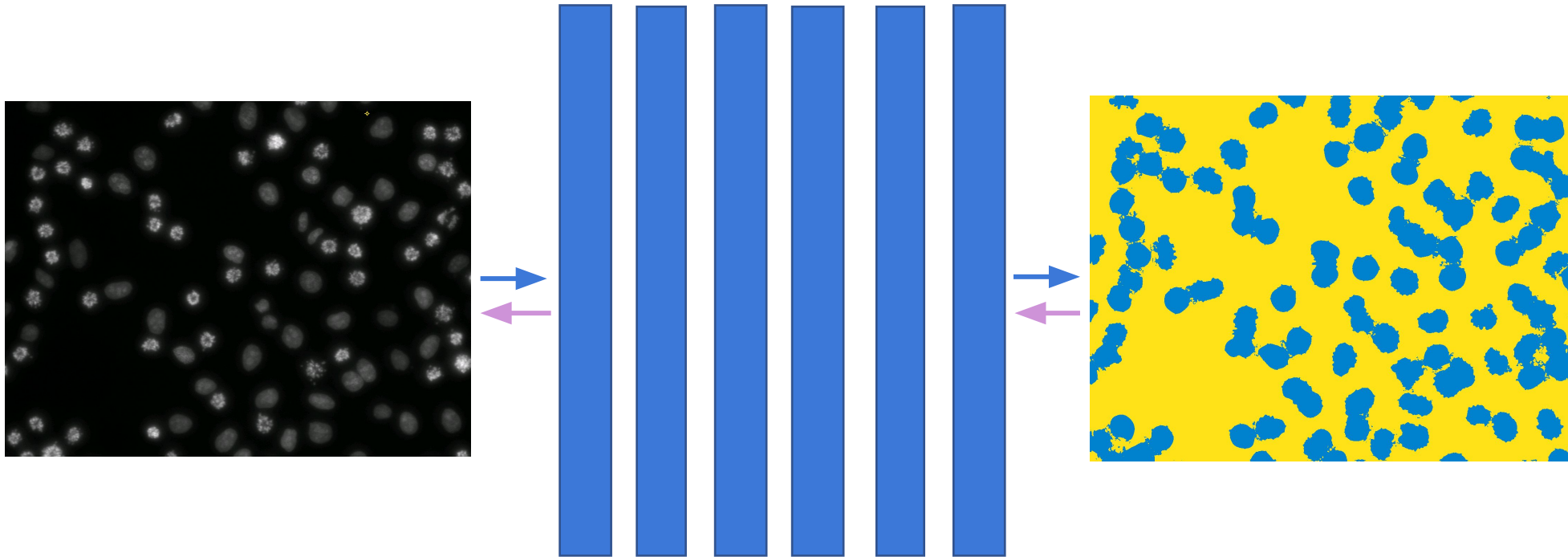
padding = 1,  
stride = 1

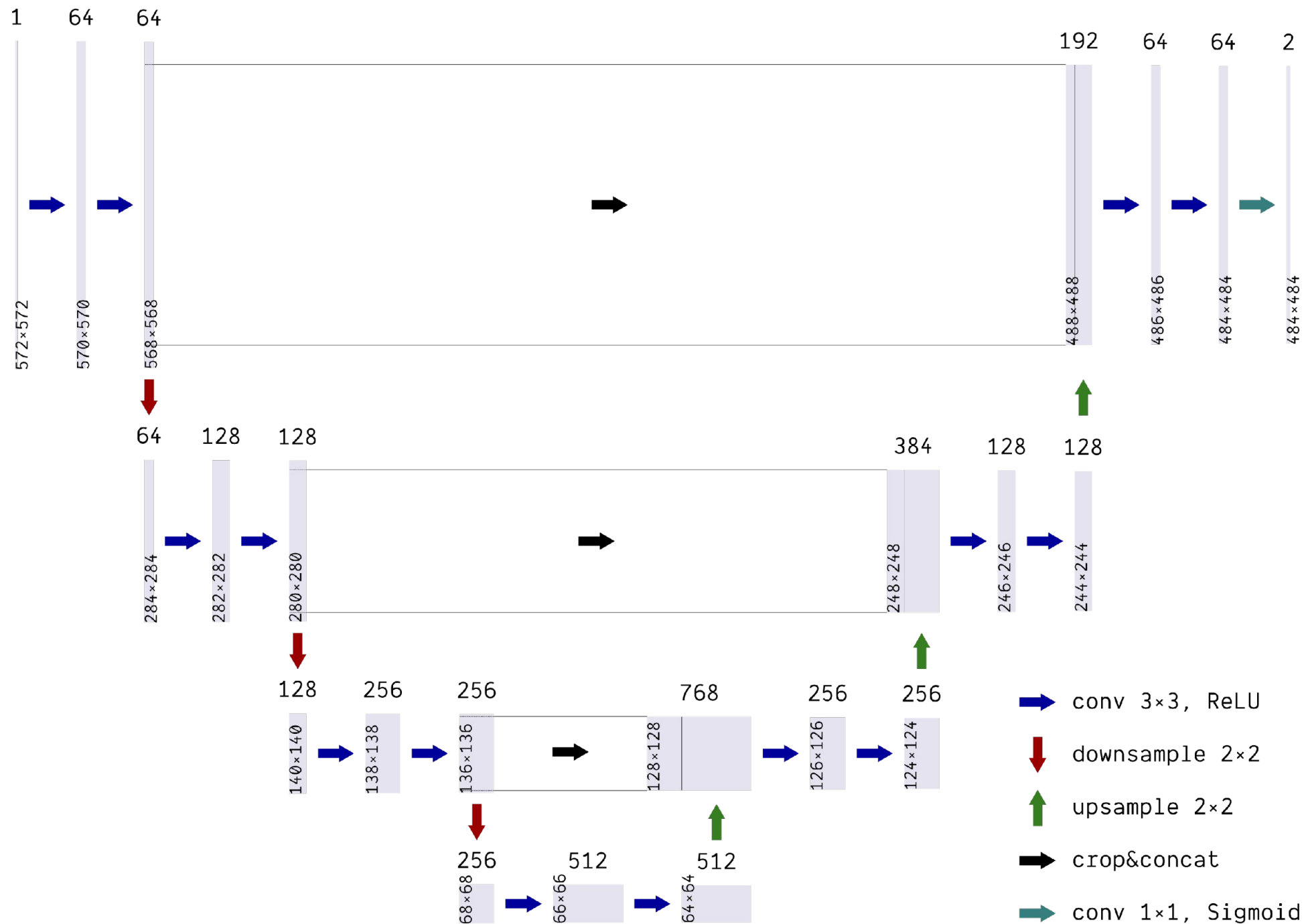
# activation functions

update feature-maps element-wise



# stacking convolutions



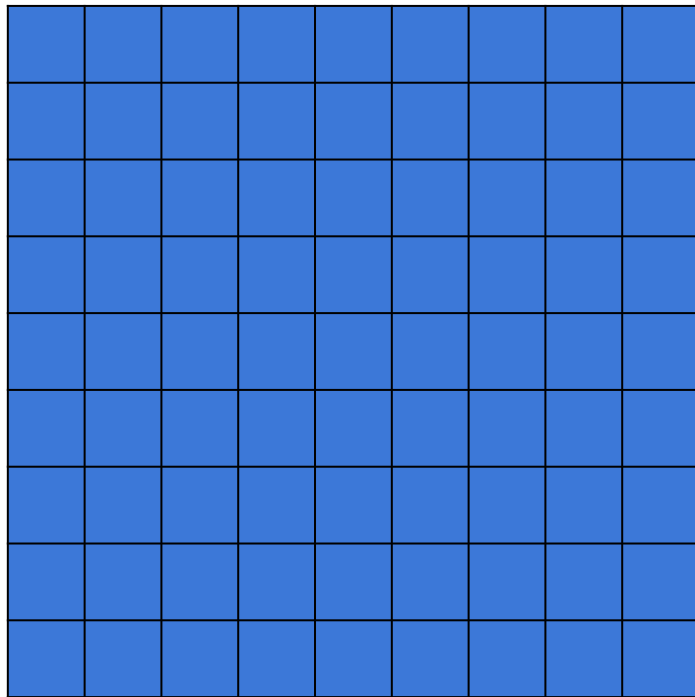




# receptive field

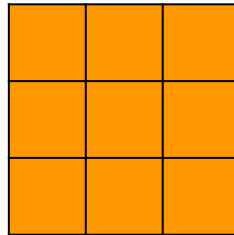
*set of input units that can influence the value of a specific output unit.*

## convolution

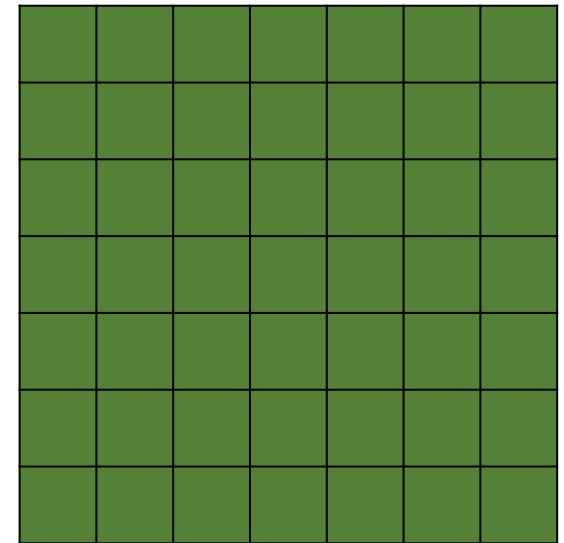


input image

\*



weights

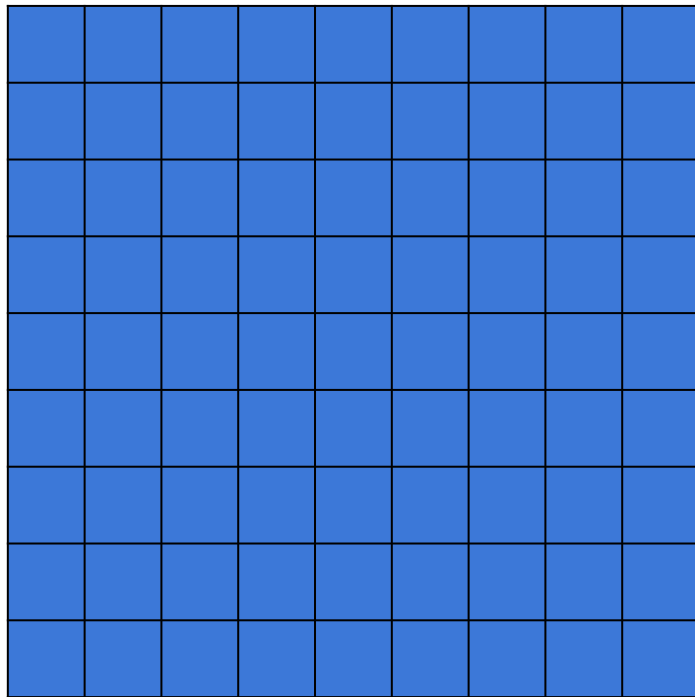


feature map

# receptive field

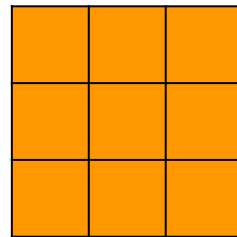
*set of input units that can influence the value of a specific output unit.*

## convolution

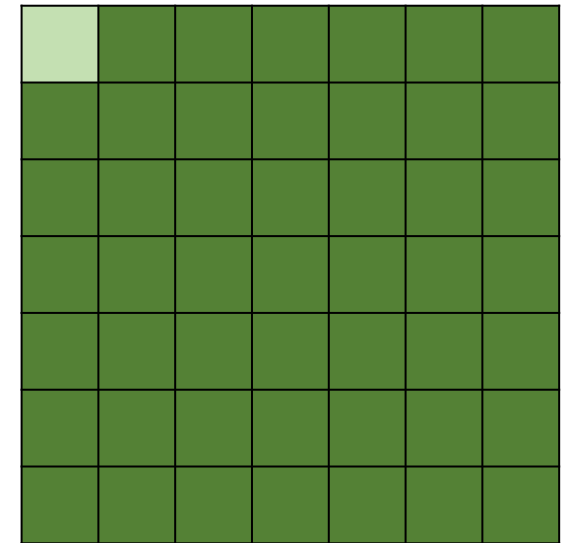


input image

\*



weights

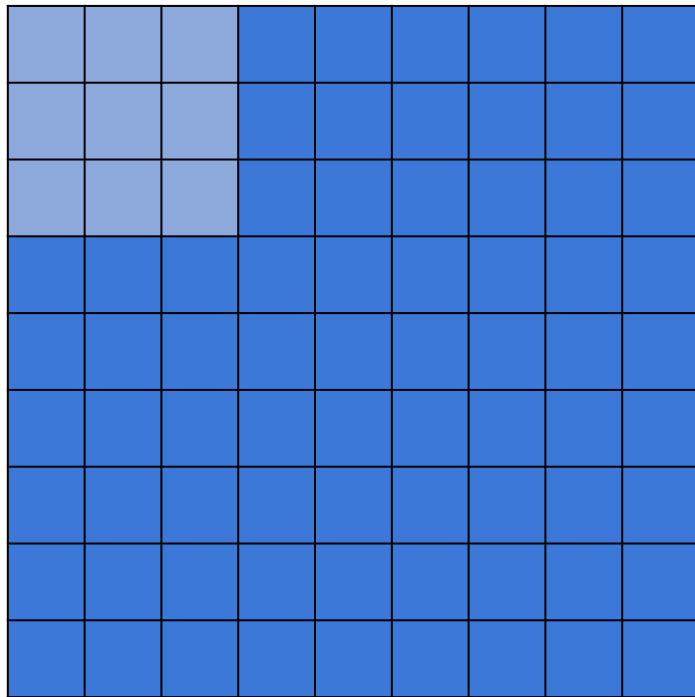


feature map

# receptive field

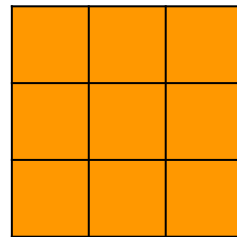
*set of input units that can influence the value of a specific output unit.*

## convolution

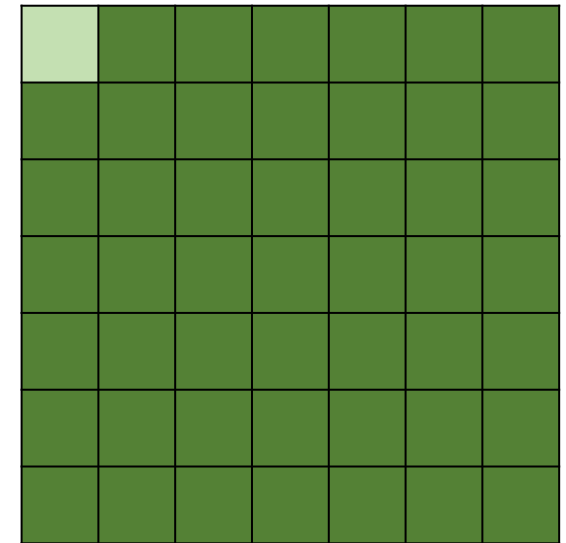


input image

\*



weights

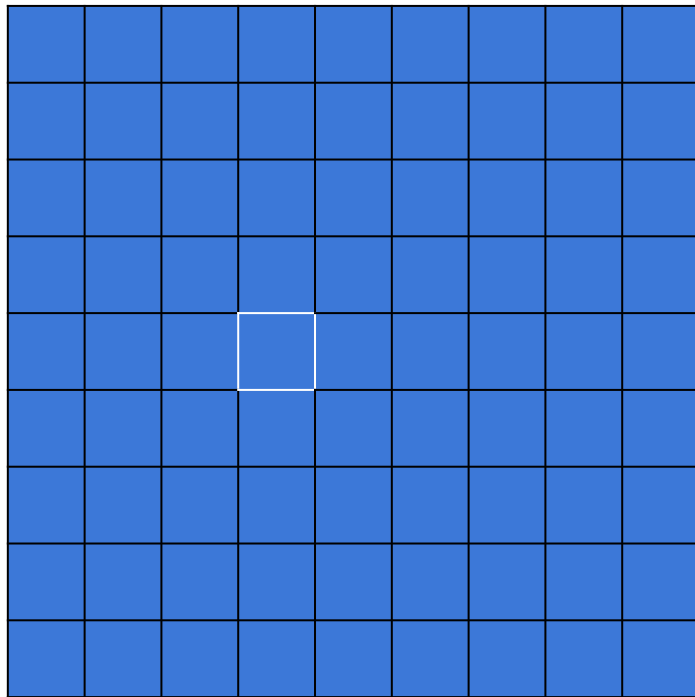


feature map

# receptive field

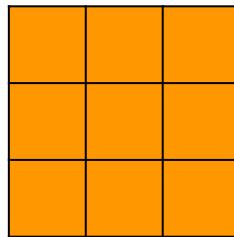
*set of input units that can influence the value of a specific output unit.*

## convolution

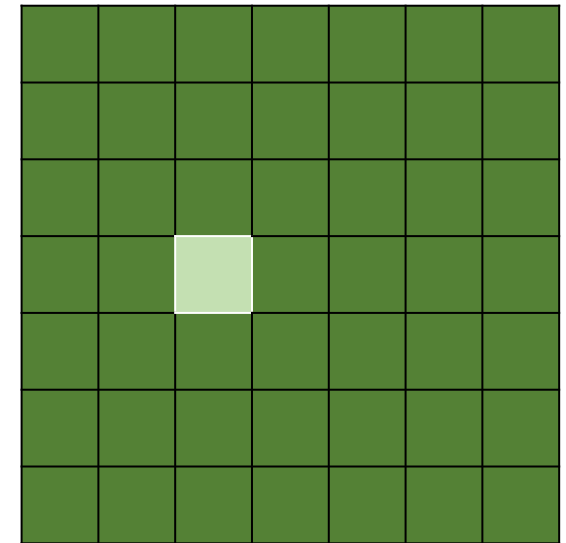


input image

\*



weights

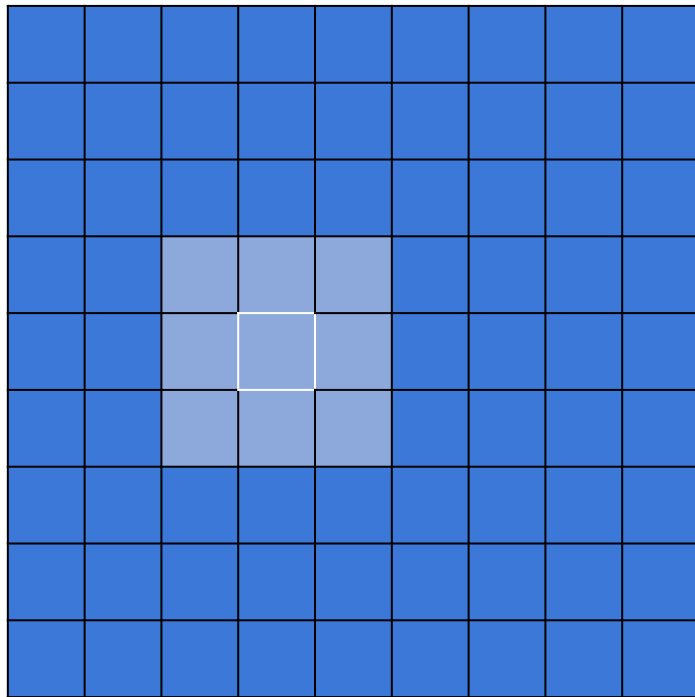


feature map

# receptive field

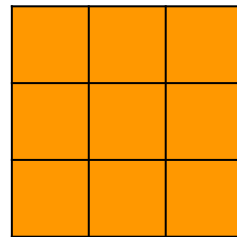
*set of input units that can influence the value of a specific output unit.*

## convolution

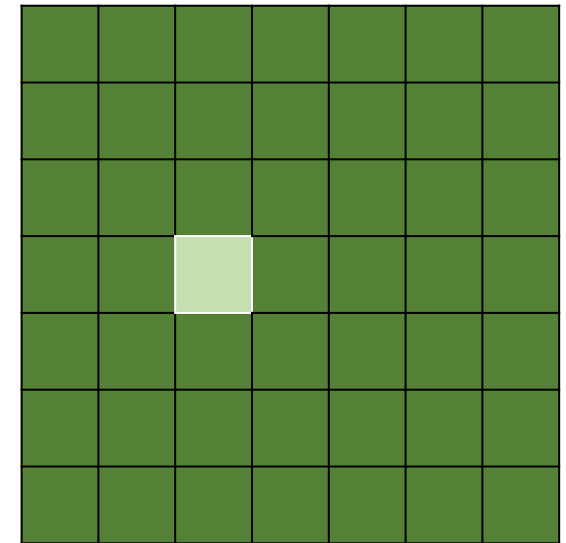


input image

\*



weights

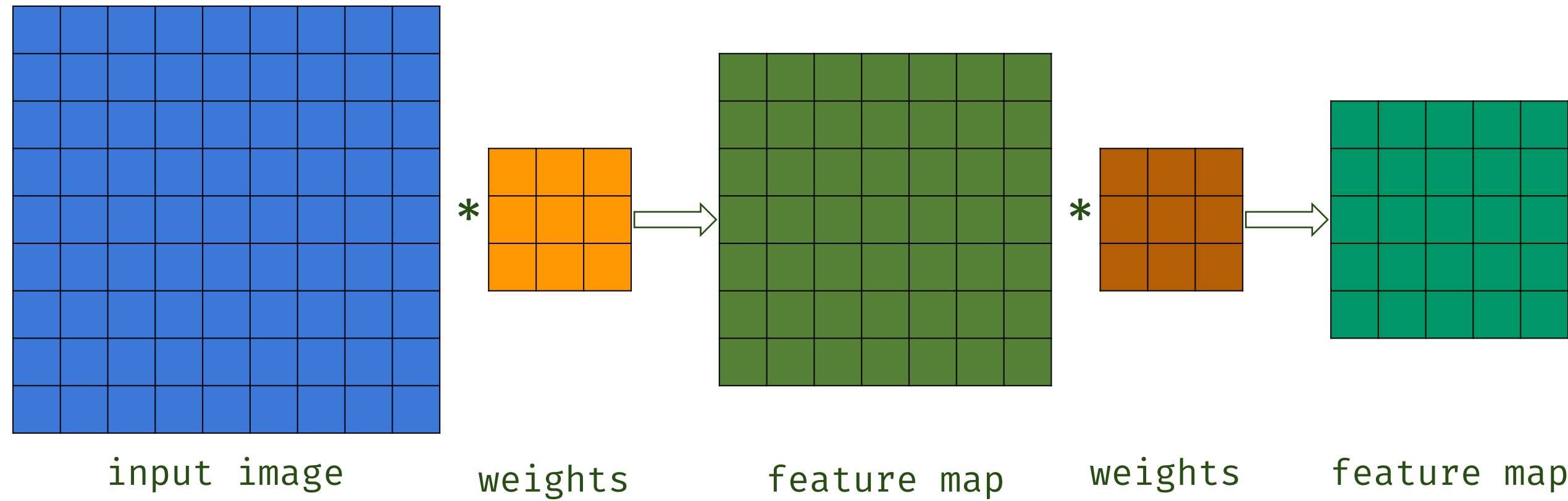


feature map

# receptive field

*set of input units that can influence the value of a specific output unit.*

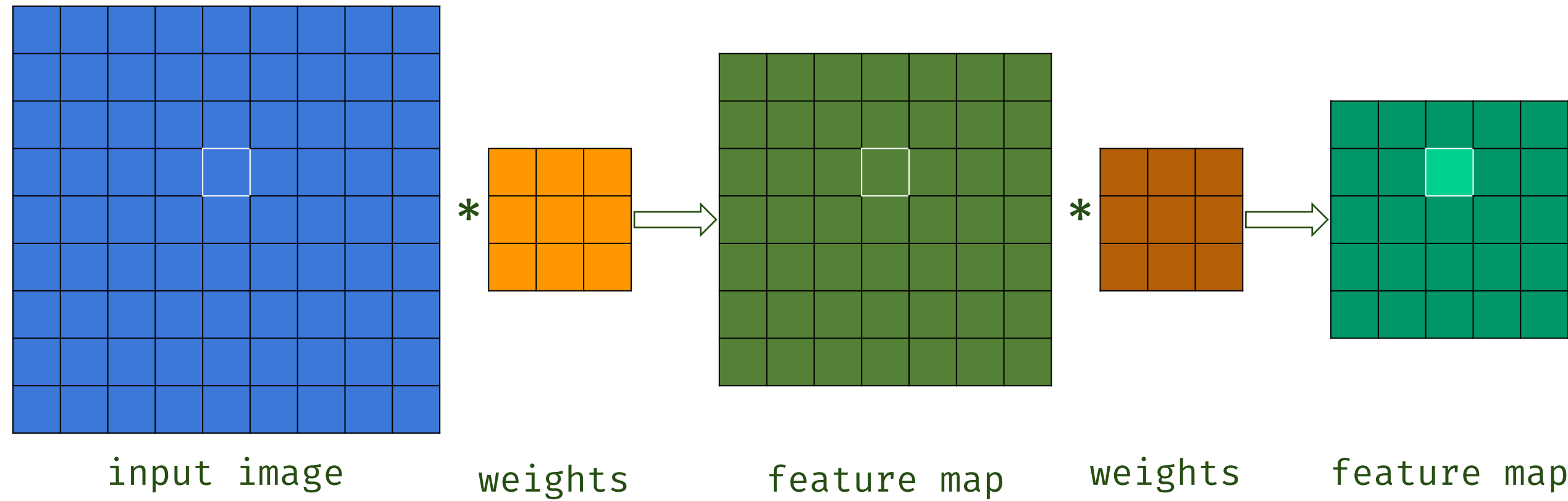
## convolution x 2



# receptive field

*set of input units that can influence the value of a specific output unit.*

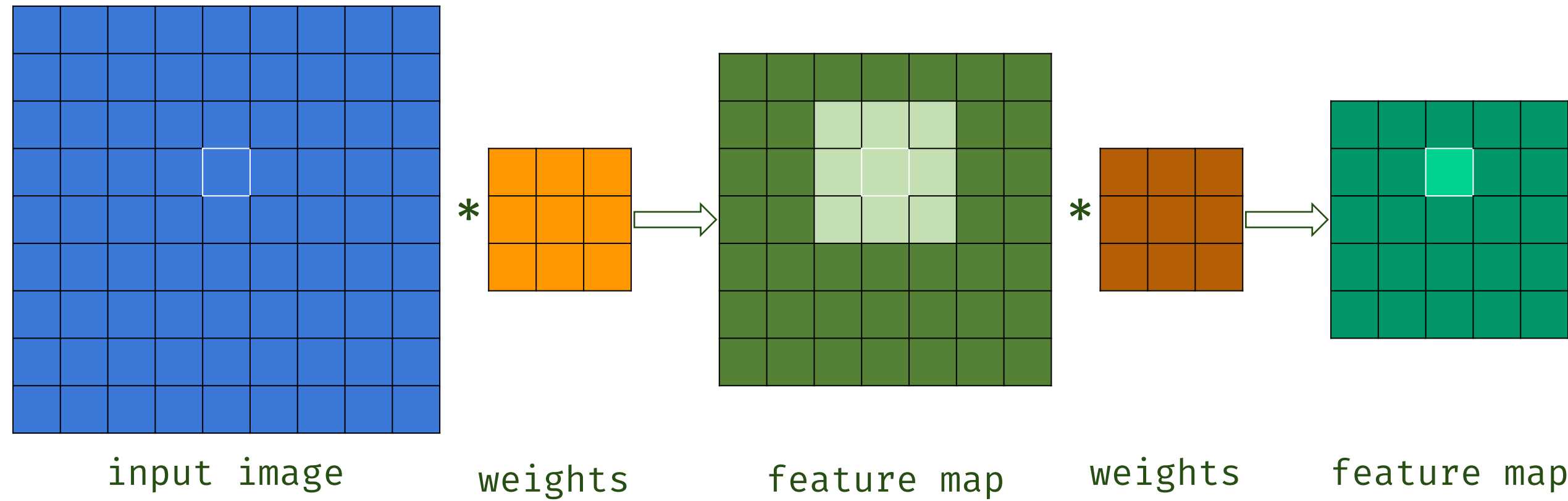
## convolution x 2



# receptive field

*set of input units that can influence the value of a specific output unit.*

## convolution x 2

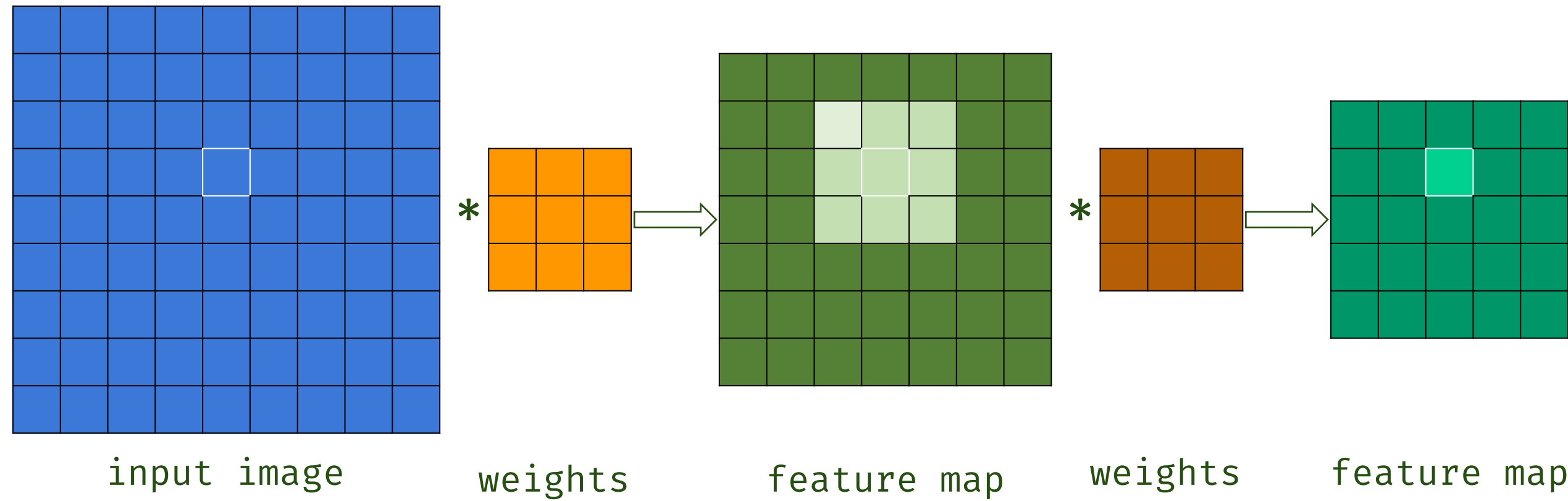




# receptive field

*set of input units that can influence the value of a specific output unit.*

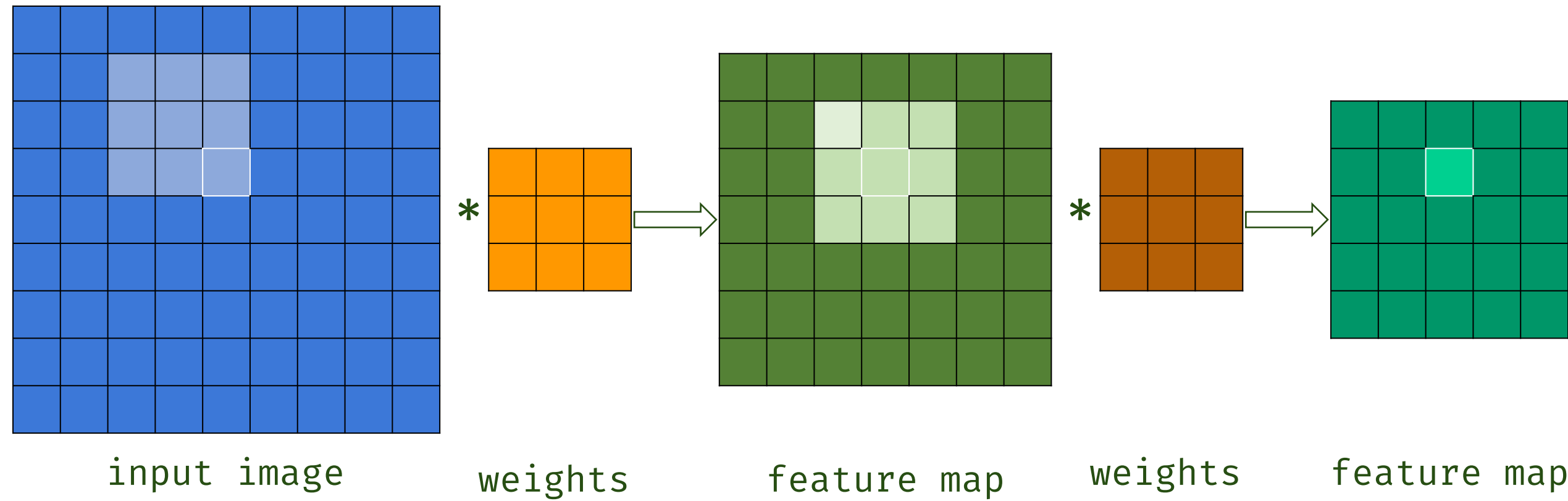
## convolution x 2



# receptive field

*set of input units that can influence the value of a specific output unit.*

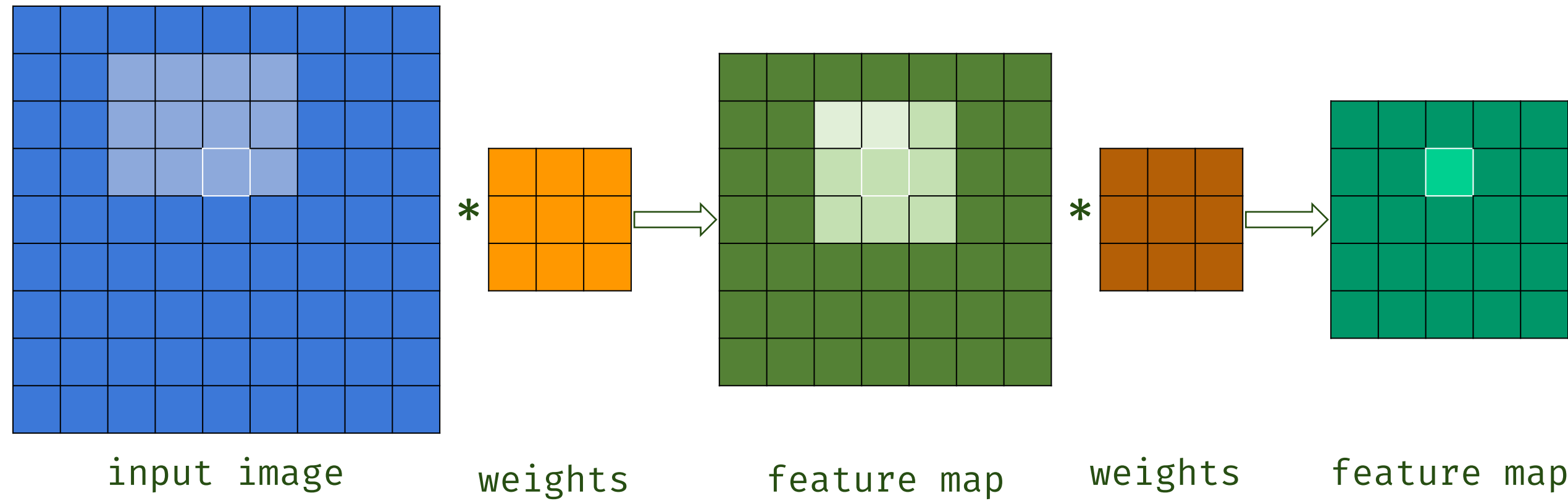
## convolution x 2



# receptive field

*set of input units that can influence the value of a specific output unit.*

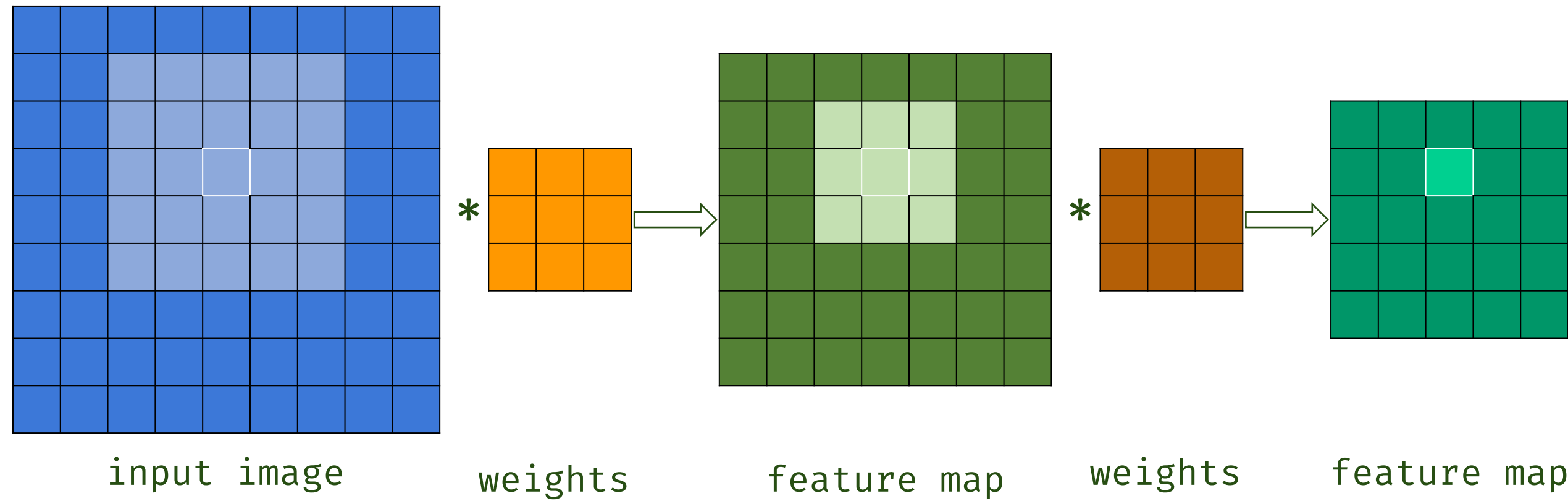
## convolution x 2



# receptive field

*set of input units that can influence the value of a specific output unit.*

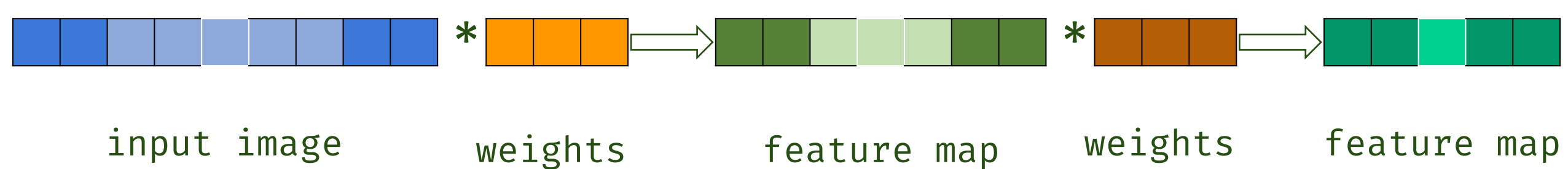
## convolution x 2



# receptive field

*set of input units that can influence the value of a specific output unit.*

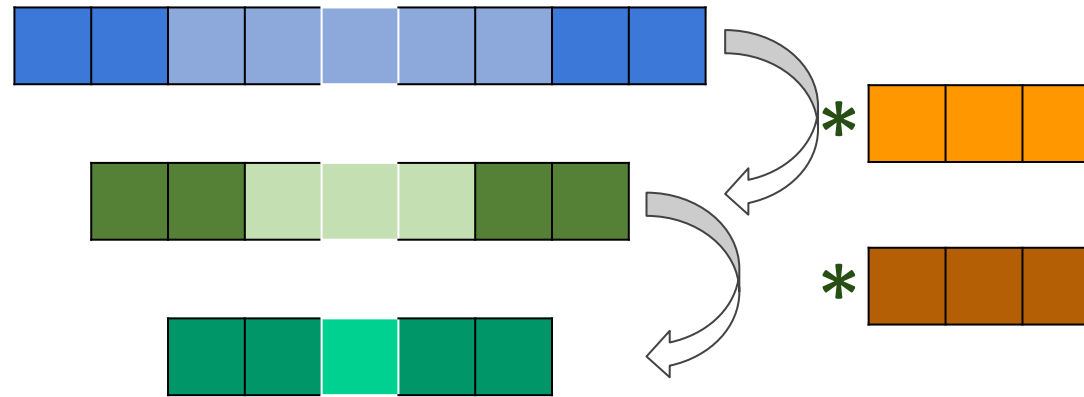
## convolution x 2



# receptive field

*set of input units that can influence the value of a specific output unit.*

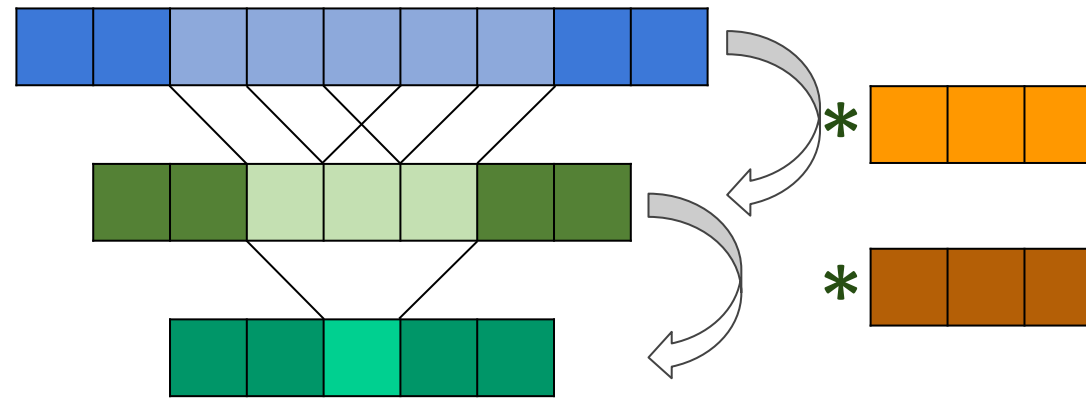
## convolution x 2



# receptive field

*set of input units that can influence the value of a specific output unit.*

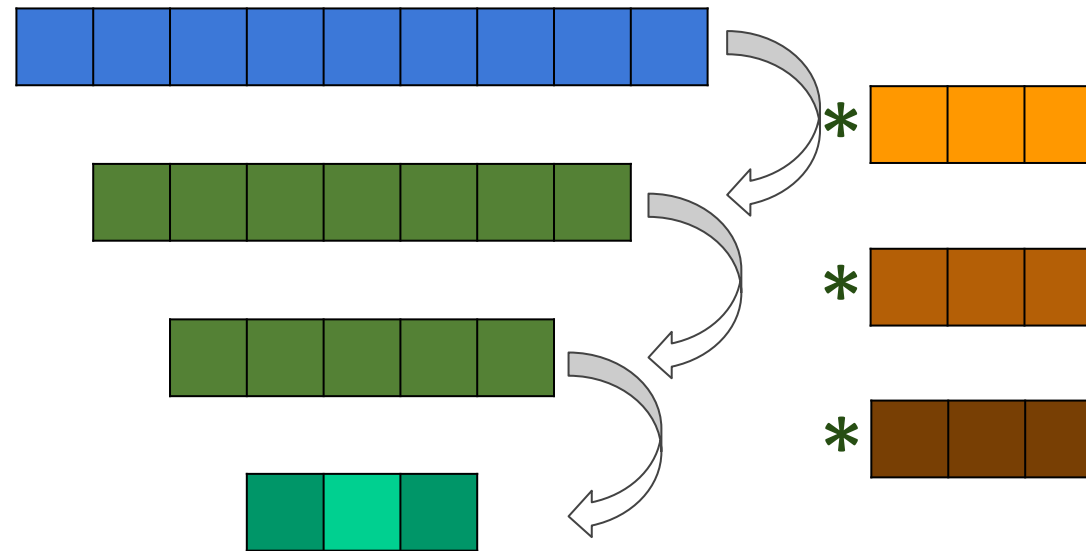
## convolution x 2



# receptive field

*set of input units that can influence the value of a specific output unit.*

## convolution x 3

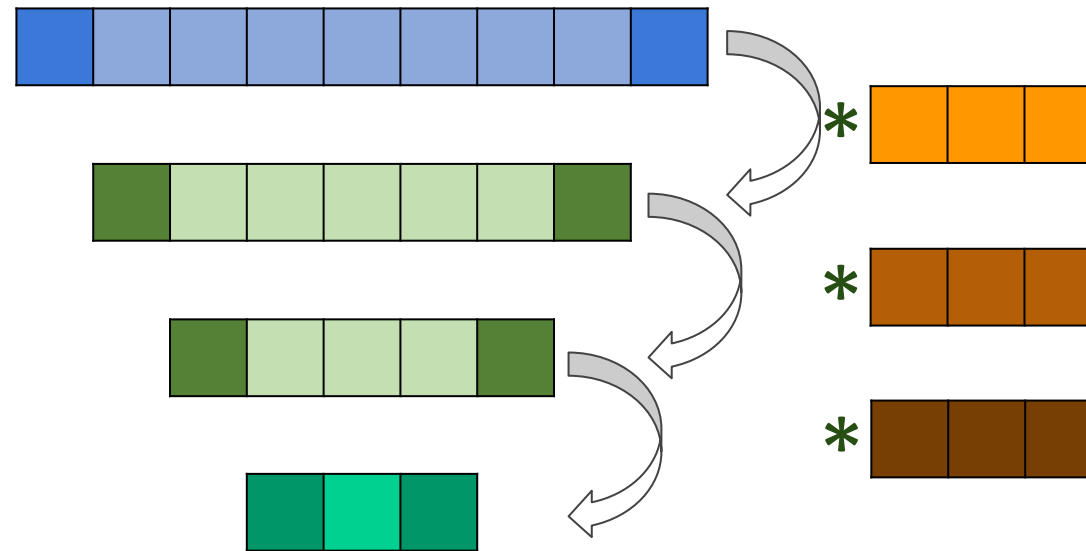




# receptive field

*set of input units that can influence the value of a specific output unit.*

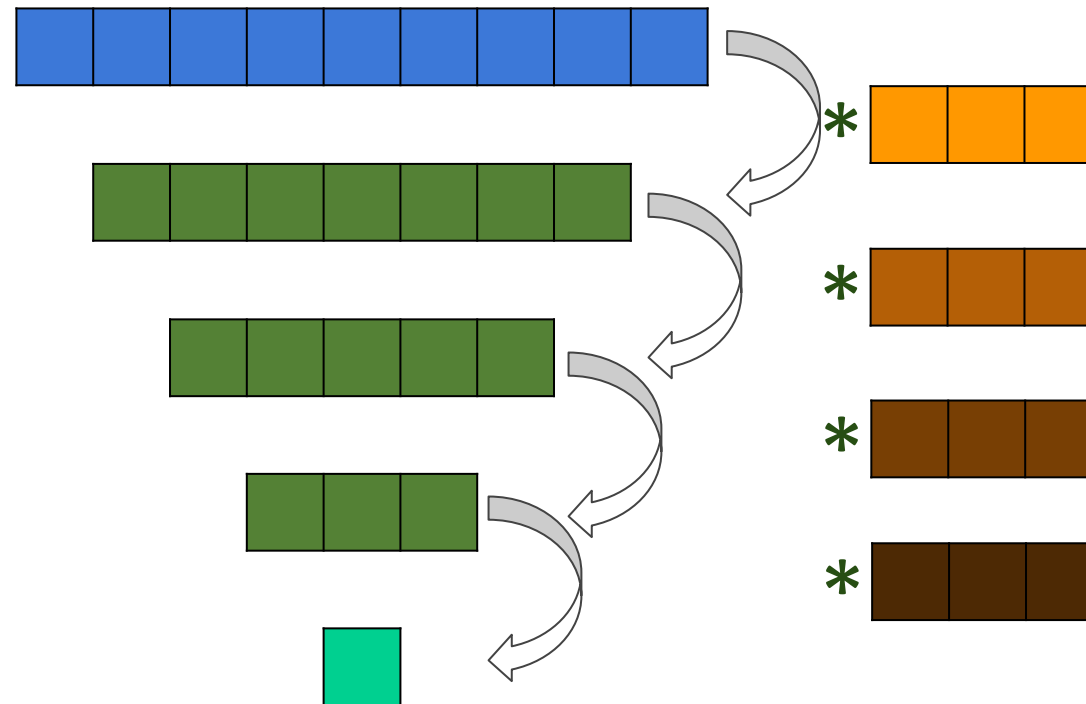
## convolution x 3



# receptive field

*set of input units that can influence the value of a specific output unit.*

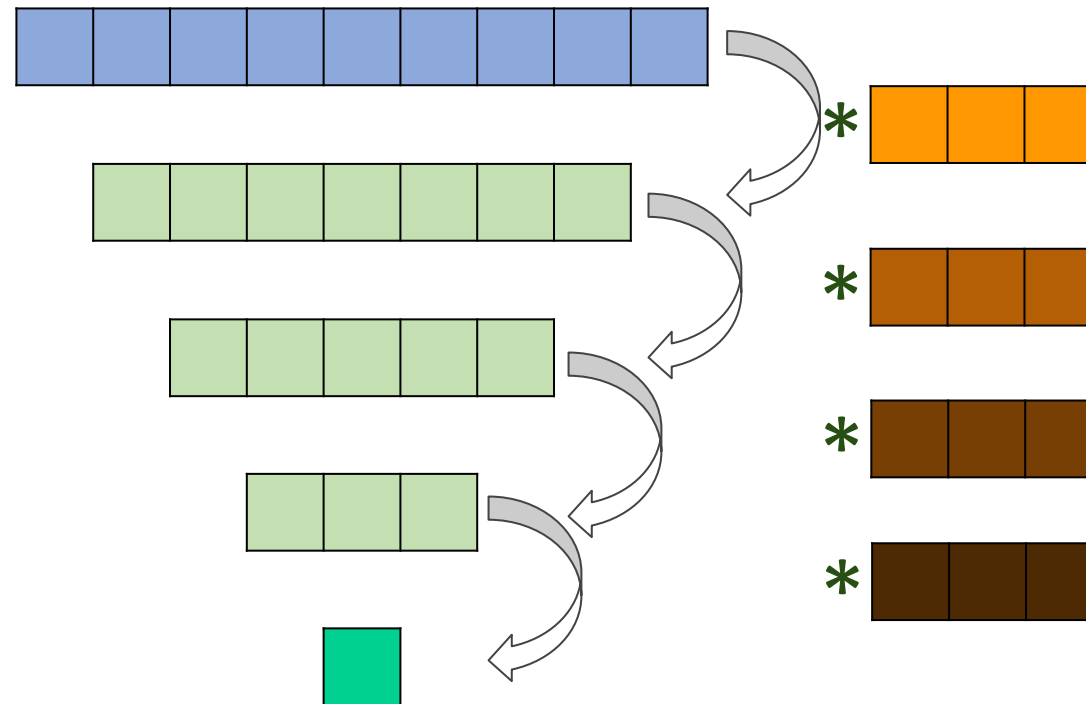
## convolution x 4



# receptive field

*set of input units that can influence the value of a specific output unit.*

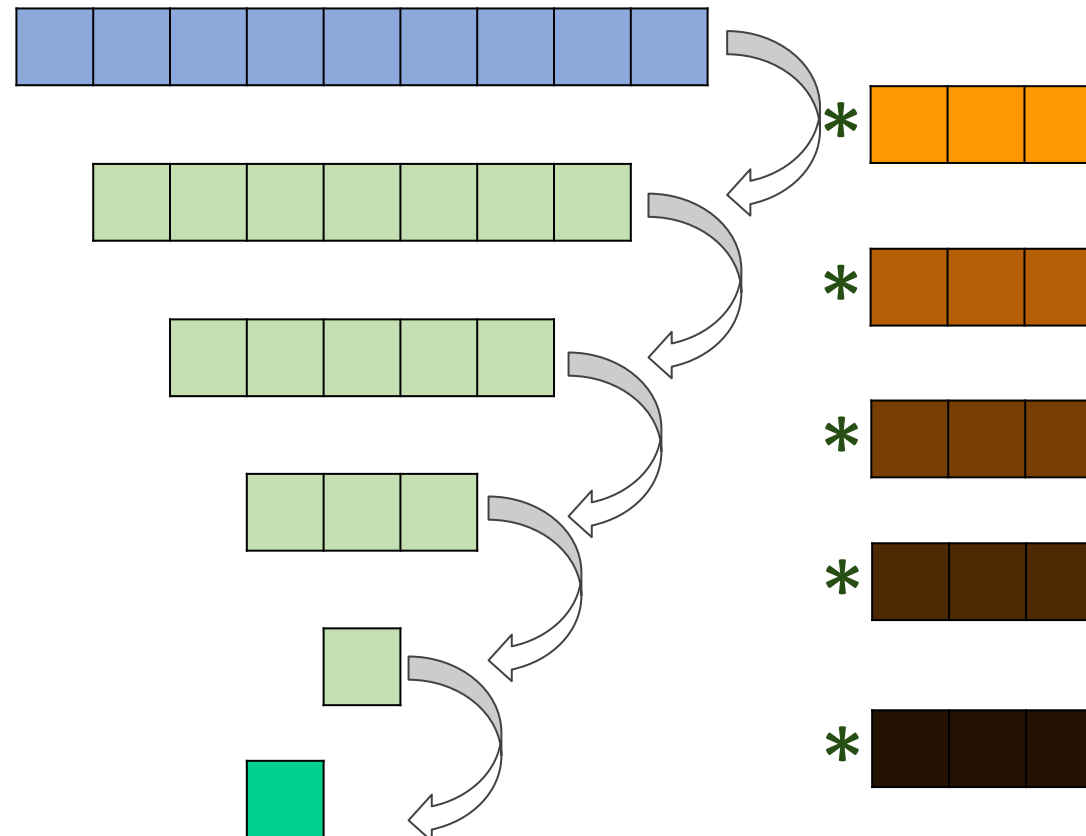
## convolution x 4



# receptive field

*set of input units that can influence the value of a specific output unit.*

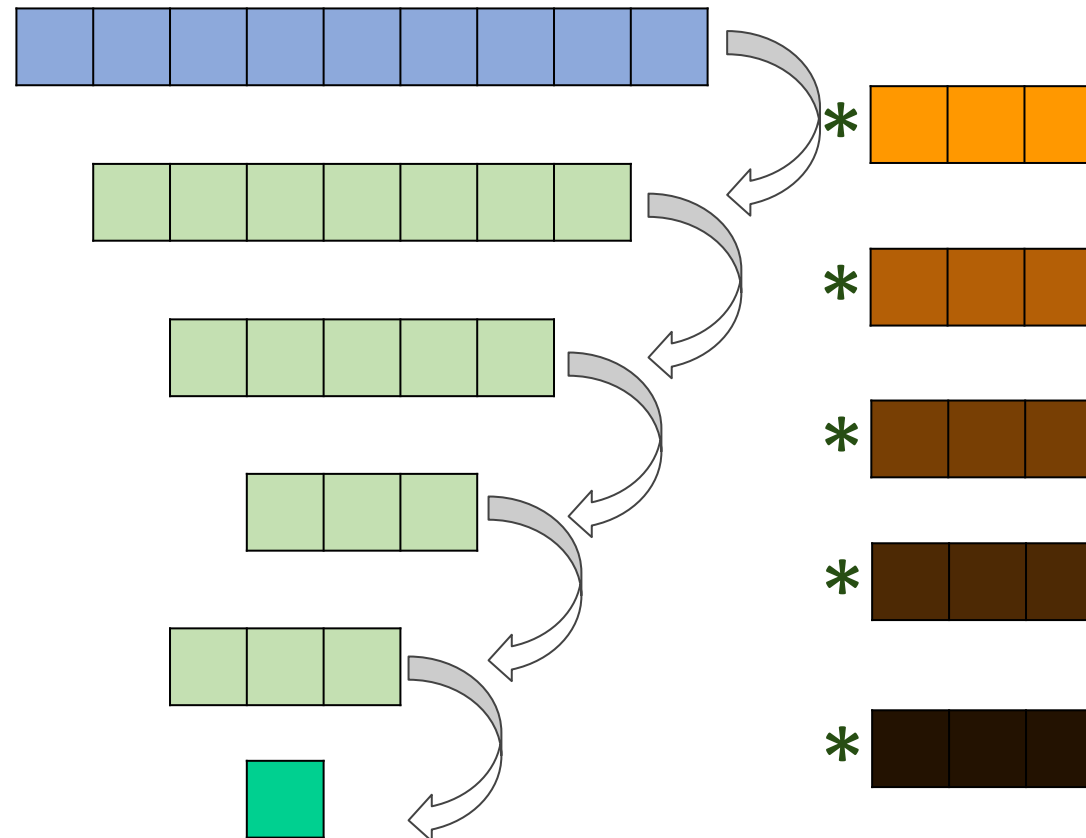
## convolution x 5



# receptive field

*set of input units that can influence the value of a specific output unit.*

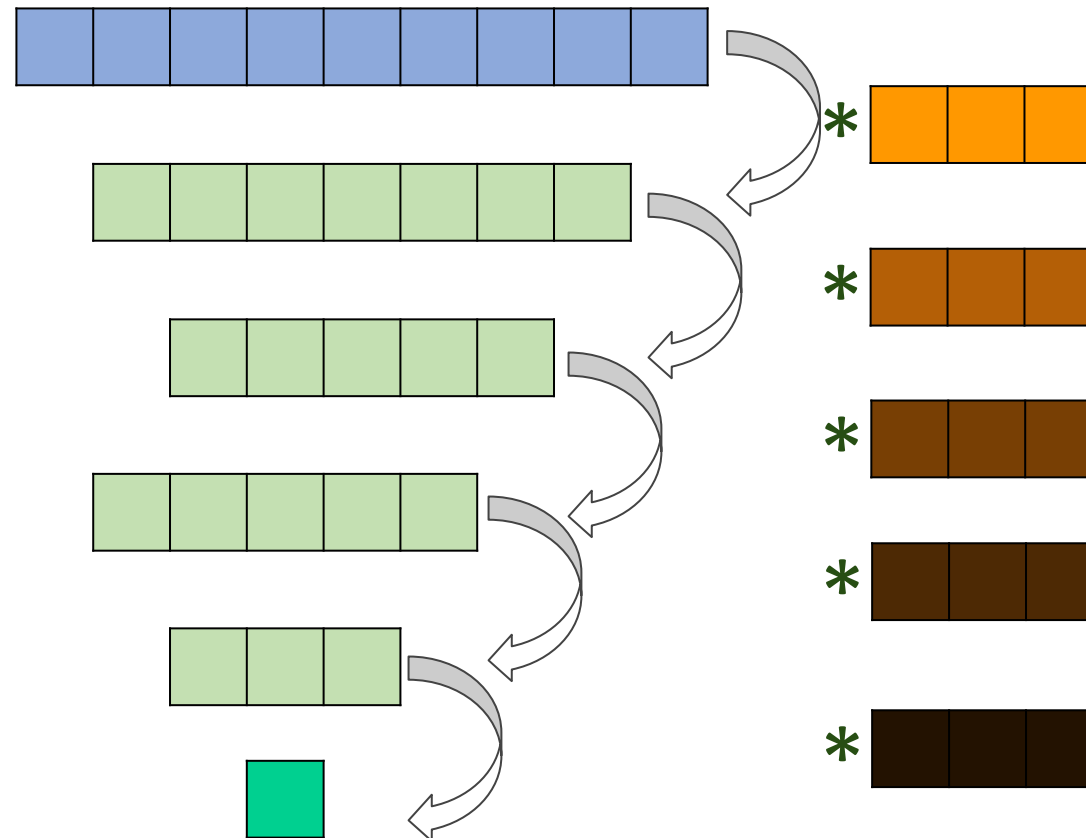
## convolution x 5



# receptive field

*set of input units that can influence the value of a specific output unit.*

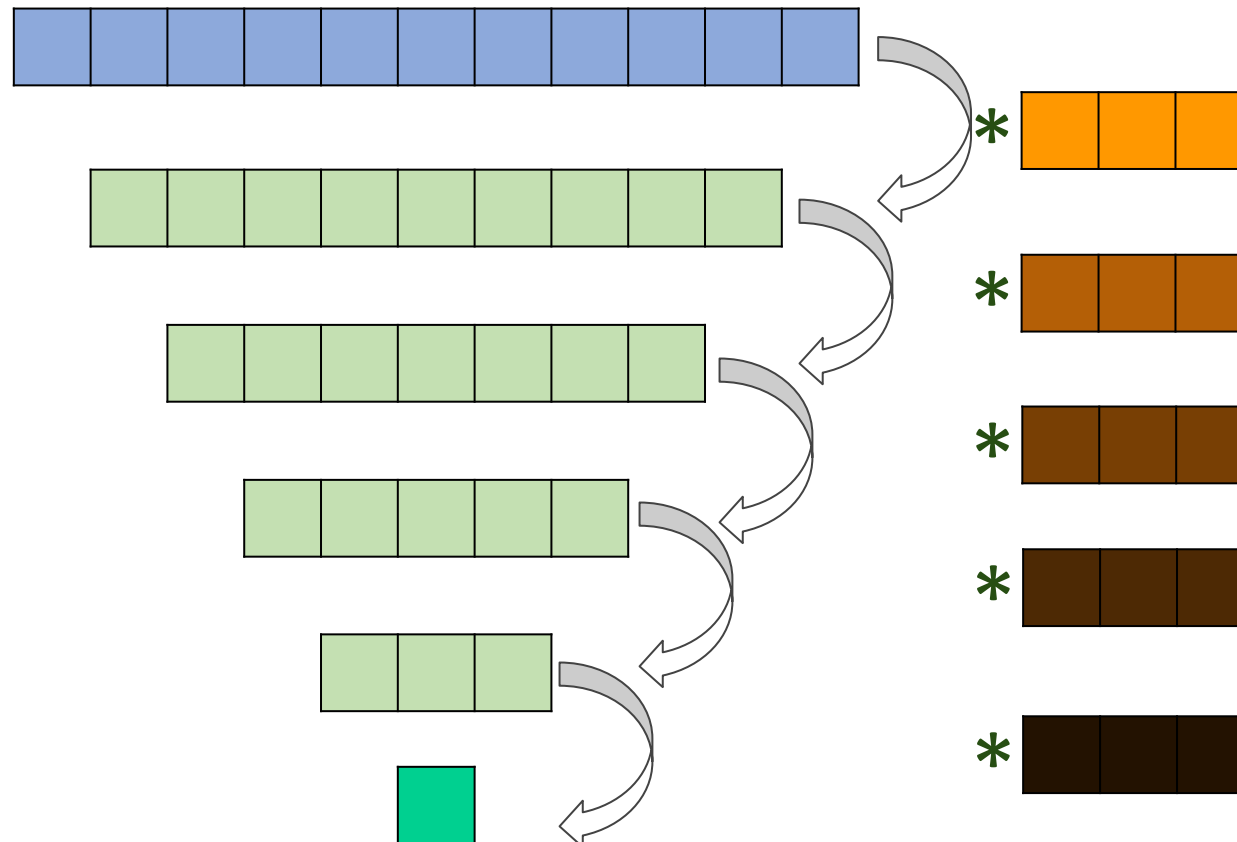
## convolution x 5



# receptive field

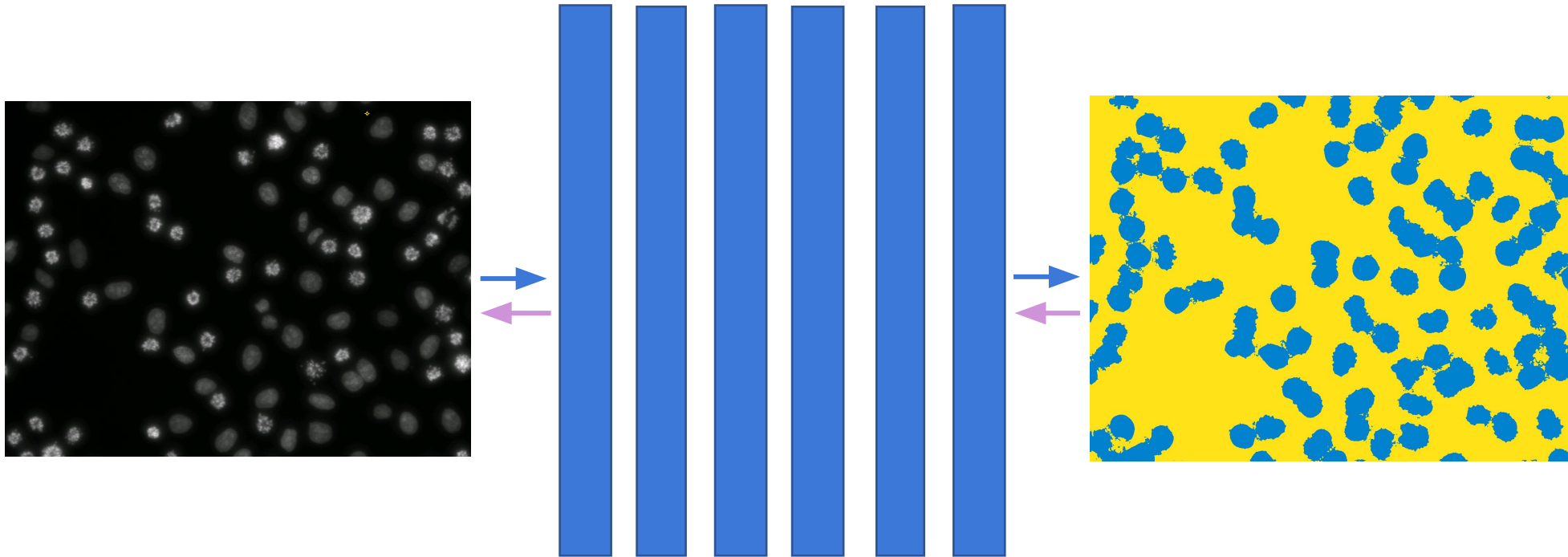
*set of input units that can influence the value of a specific output unit.*

## convolution x 5



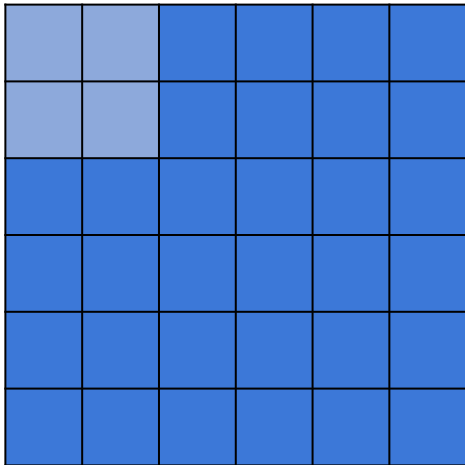
# stacking convolutions

receptive field size =  $1 + \text{number\_of\_convolutions} * (\text{kernel\_size} - 1)$





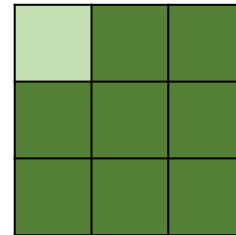
# downsampling



some layer

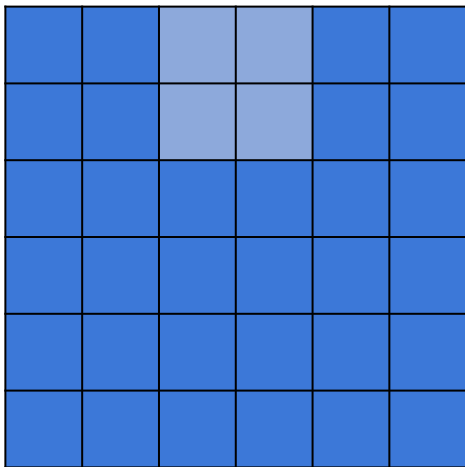
\*

max/avg  
pooling



next layer

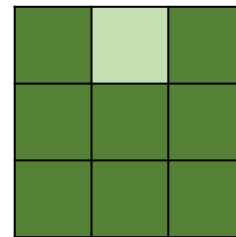
# downsampling



some layer

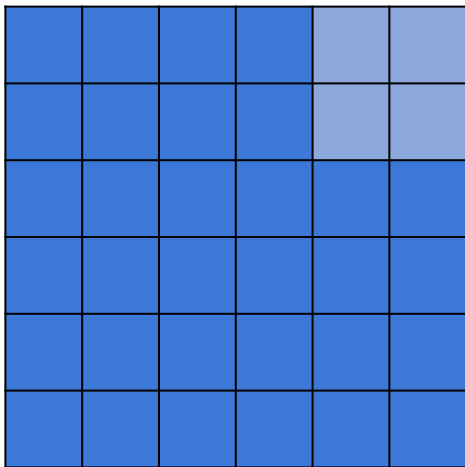
\*

max/avg  
pooling



next layer

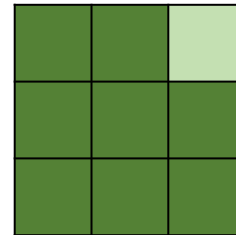
# downsampling



some layer

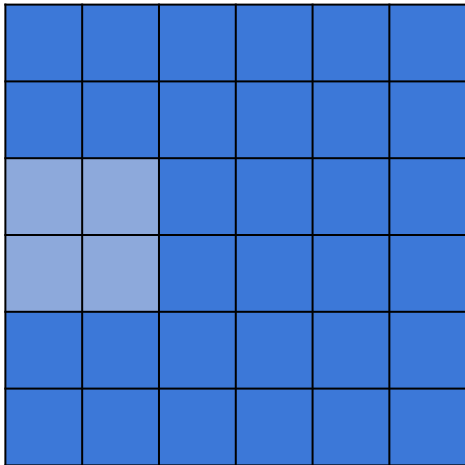
\*

max/avg  
pooling



next layer

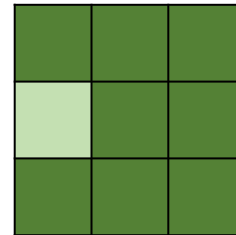
# downsampling



some layer

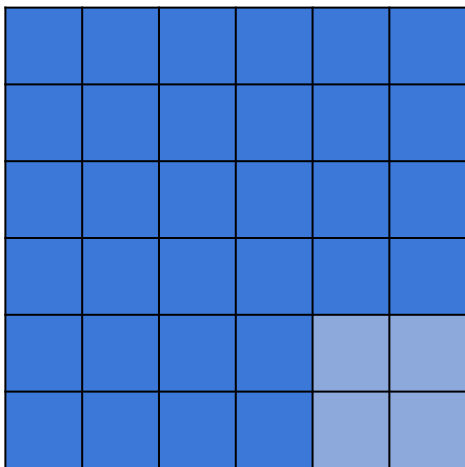
\*

max/avg  
pooling



next layer

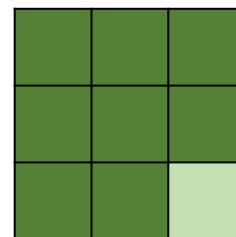
# downsampling



some layer

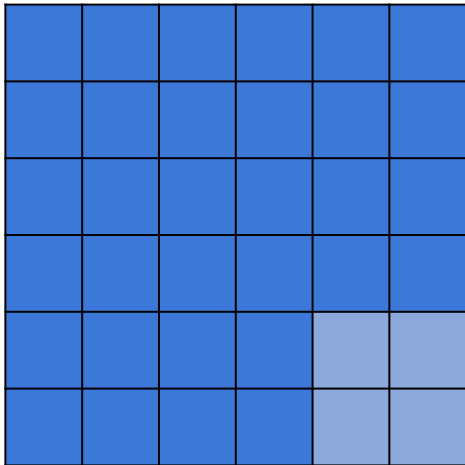
\*

max/avg  
pooling



next layer

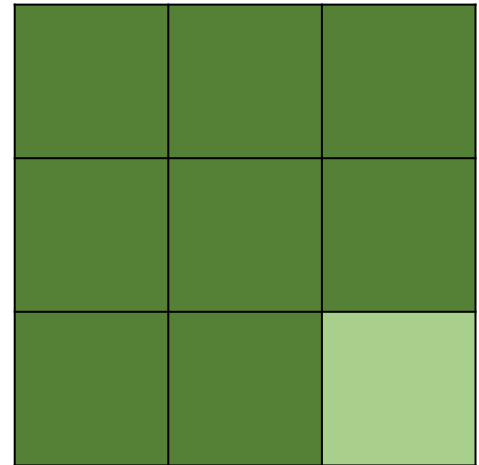
# downsampling



some layer

\*

max/avg  
pooling

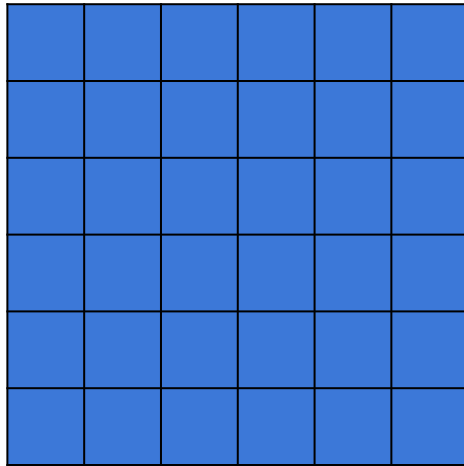


next layer

# receptive field

*set of input units that can influence the value of a specific output unit.*

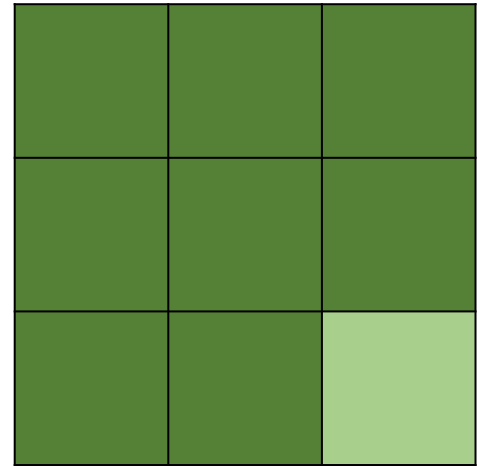
## max pooling



some layer

\*

max/avg  
pooling

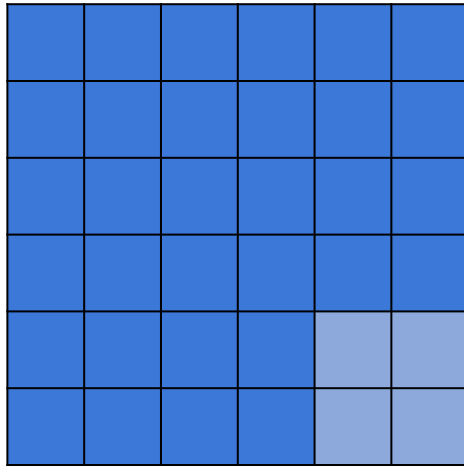


next layer

# receptive field

*set of input units that can influence the value of a specific output unit.*

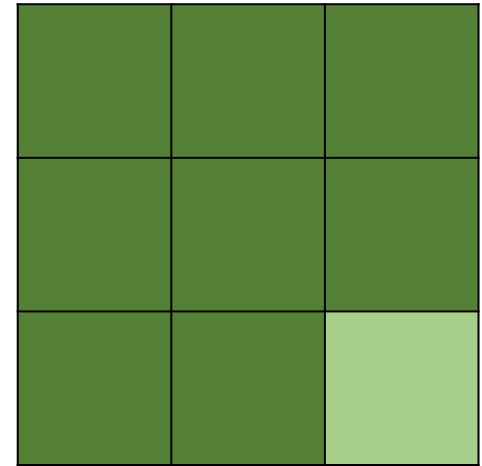
## max pooling



some layer

\*

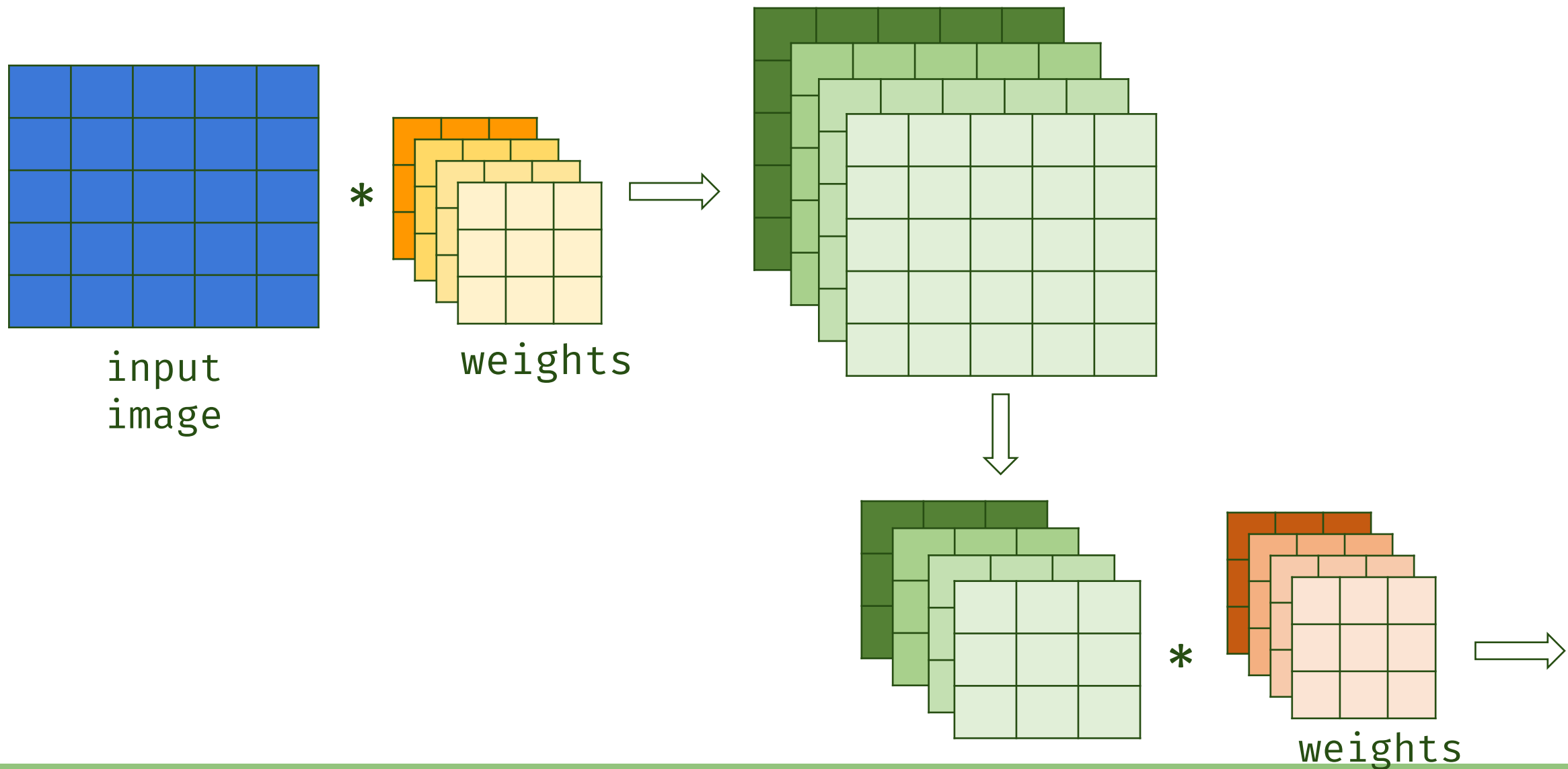
max/avg  
pooling

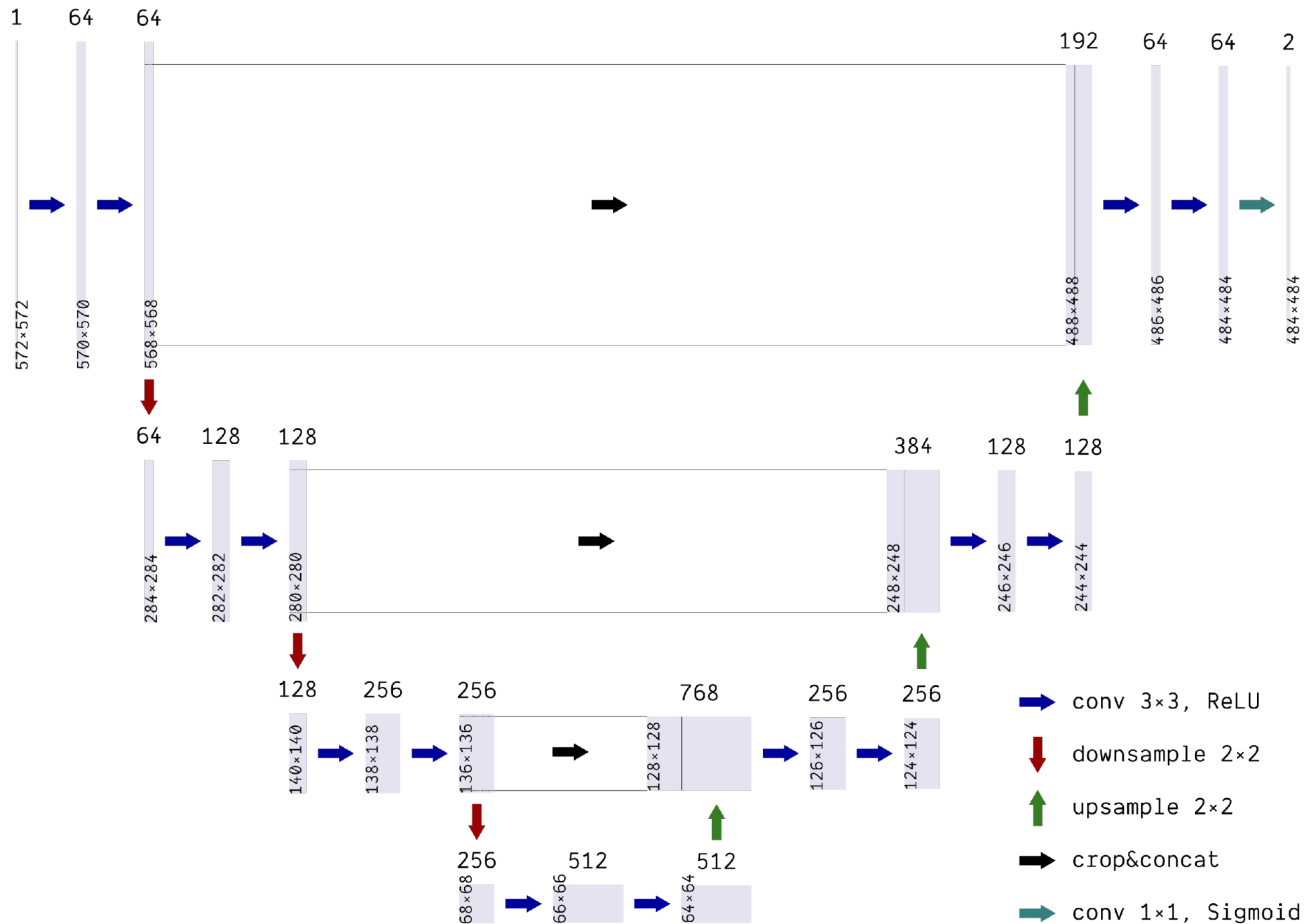


next layer



# building an encoder

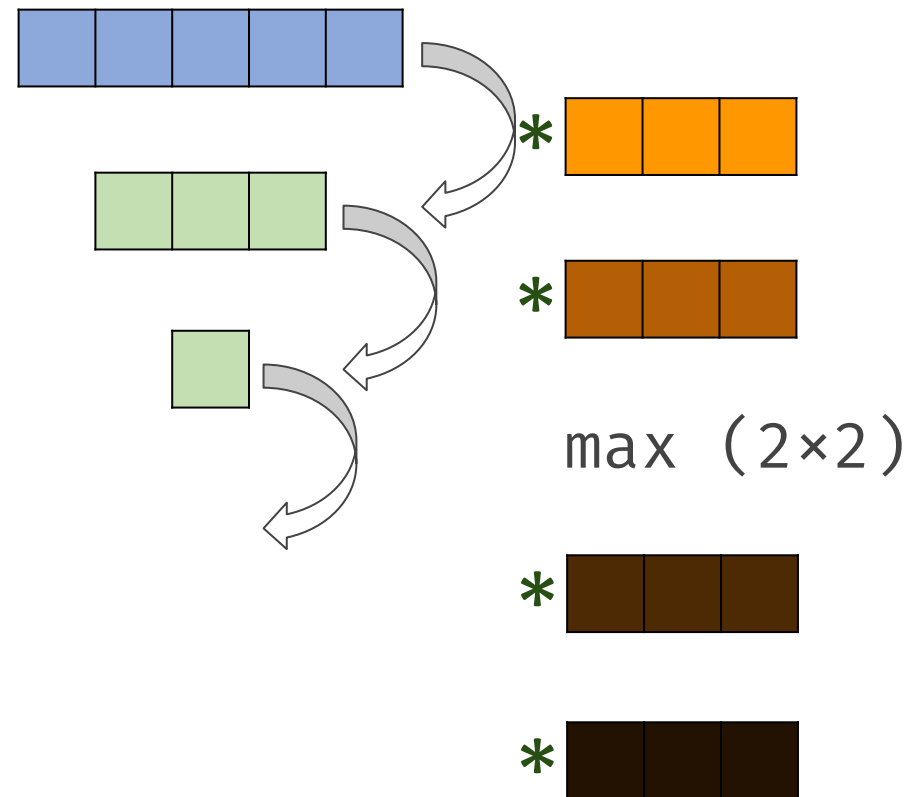




# receptive field

*set of input units that can influence the value of a specific output unit.*

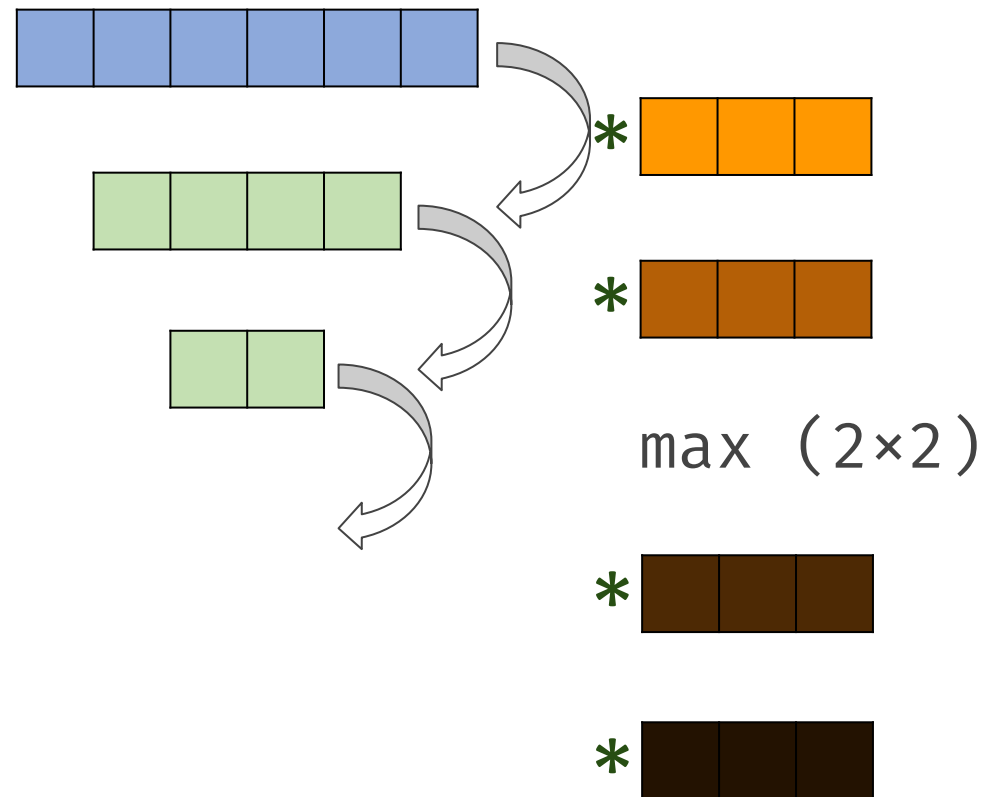
**convolution x 2 + max pooling + convolution x 2**



# receptive field

*set of input units that can influence the value of a specific output unit.*

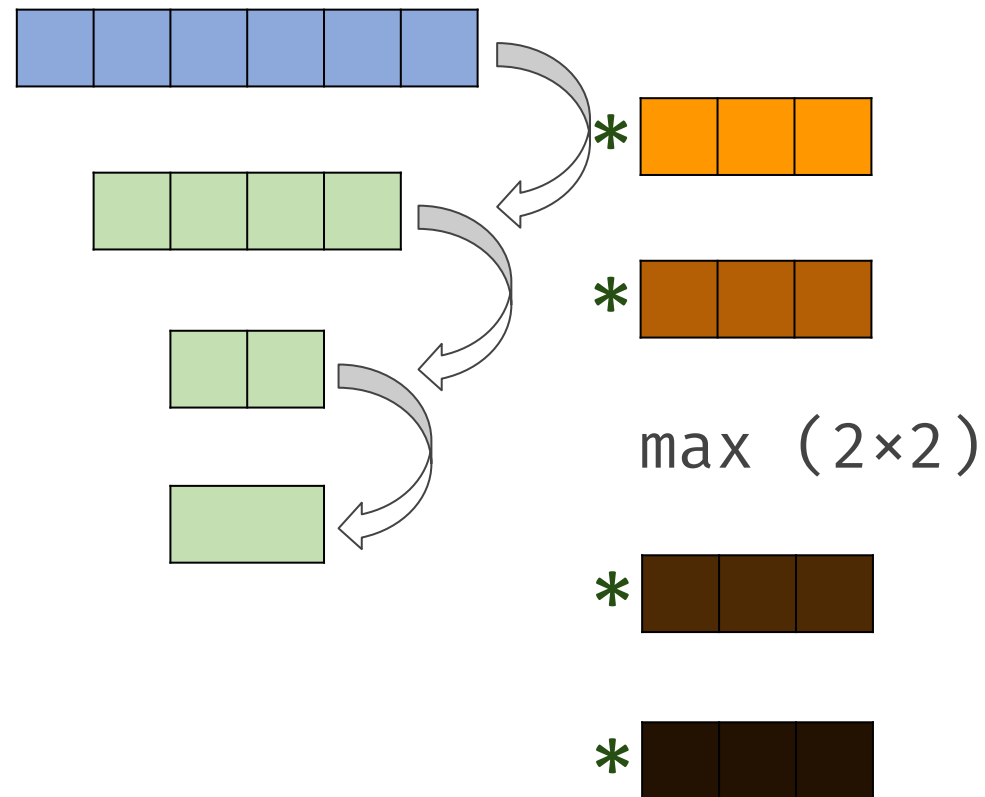
**convolution x 2 + max pooling + convolution x 2**



# receptive field

*set of input units that can influence the value of a specific output unit.*

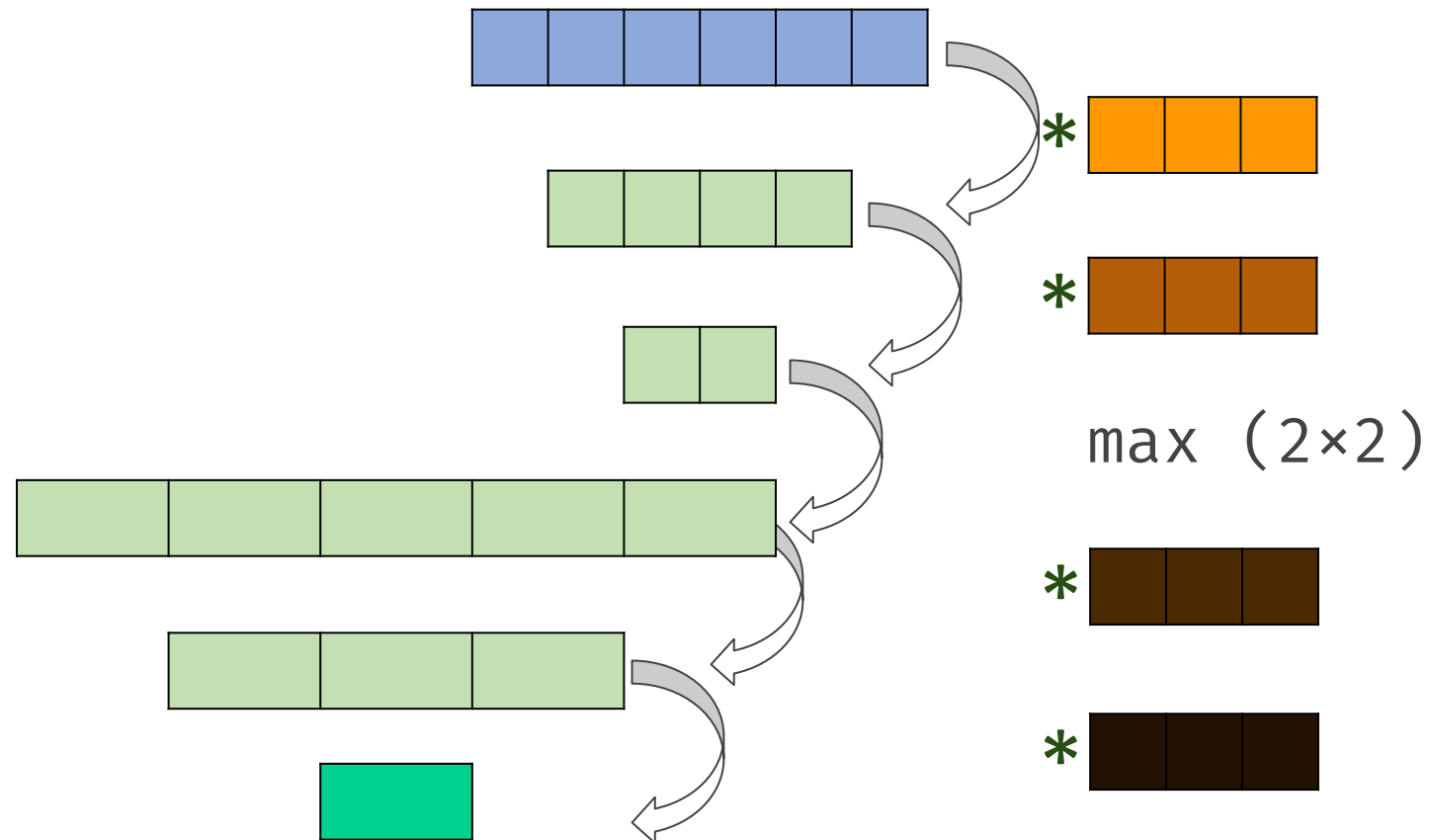
**convolution x 2 + max pooling + convolution x 2**



# receptive field

*set of input units that can influence the value of a specific output unit.*

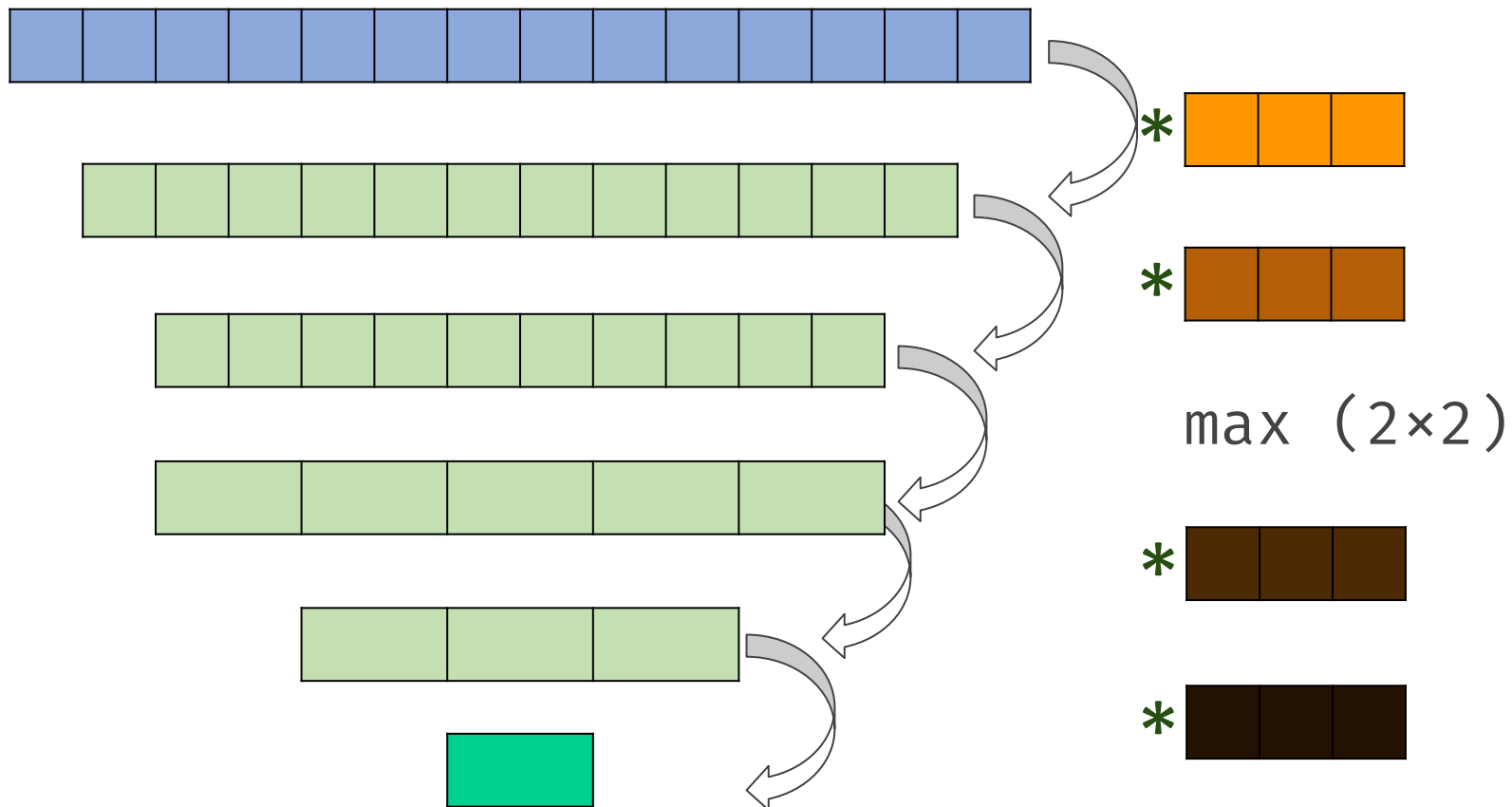
**convolution x 2 + max pooling + convolution x 2**



# receptive field

*set of input units that can influence the value of a specific output unit.*

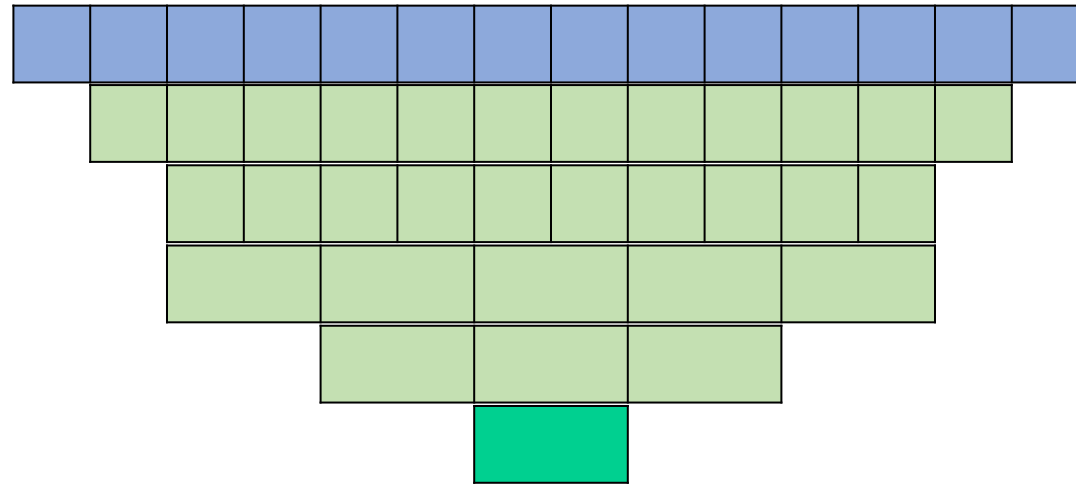
**convolution x 2 + max pooling + convolution x 2**



# receptive field

*set of input units that can influence the value of a specific output unit.*

**convolution x 2 + max pooling + convolution x 2**

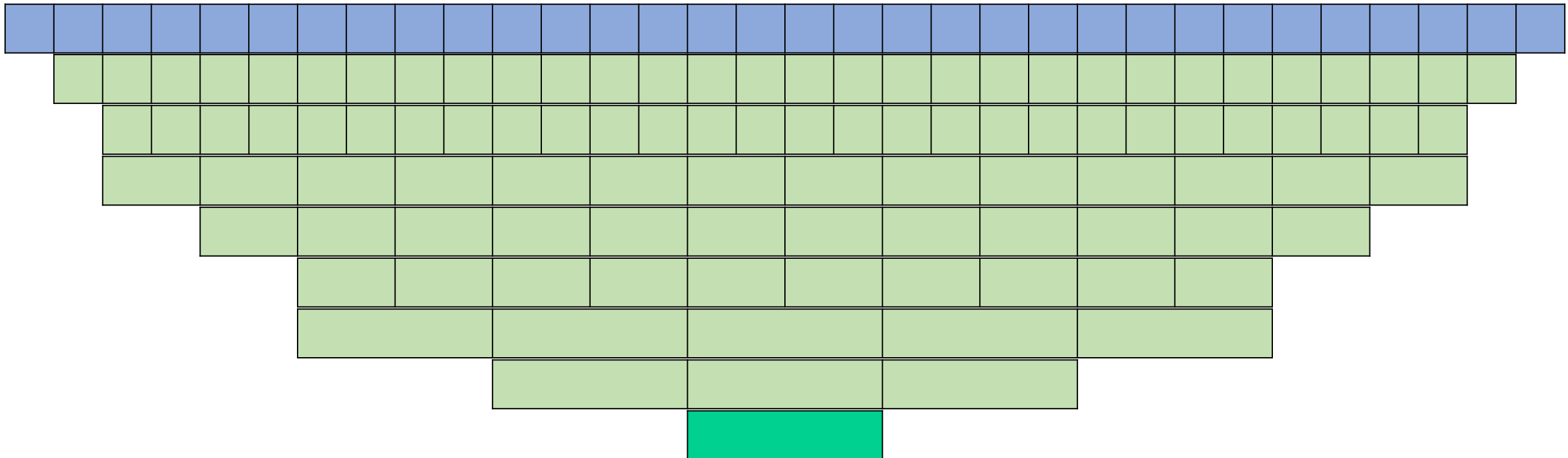


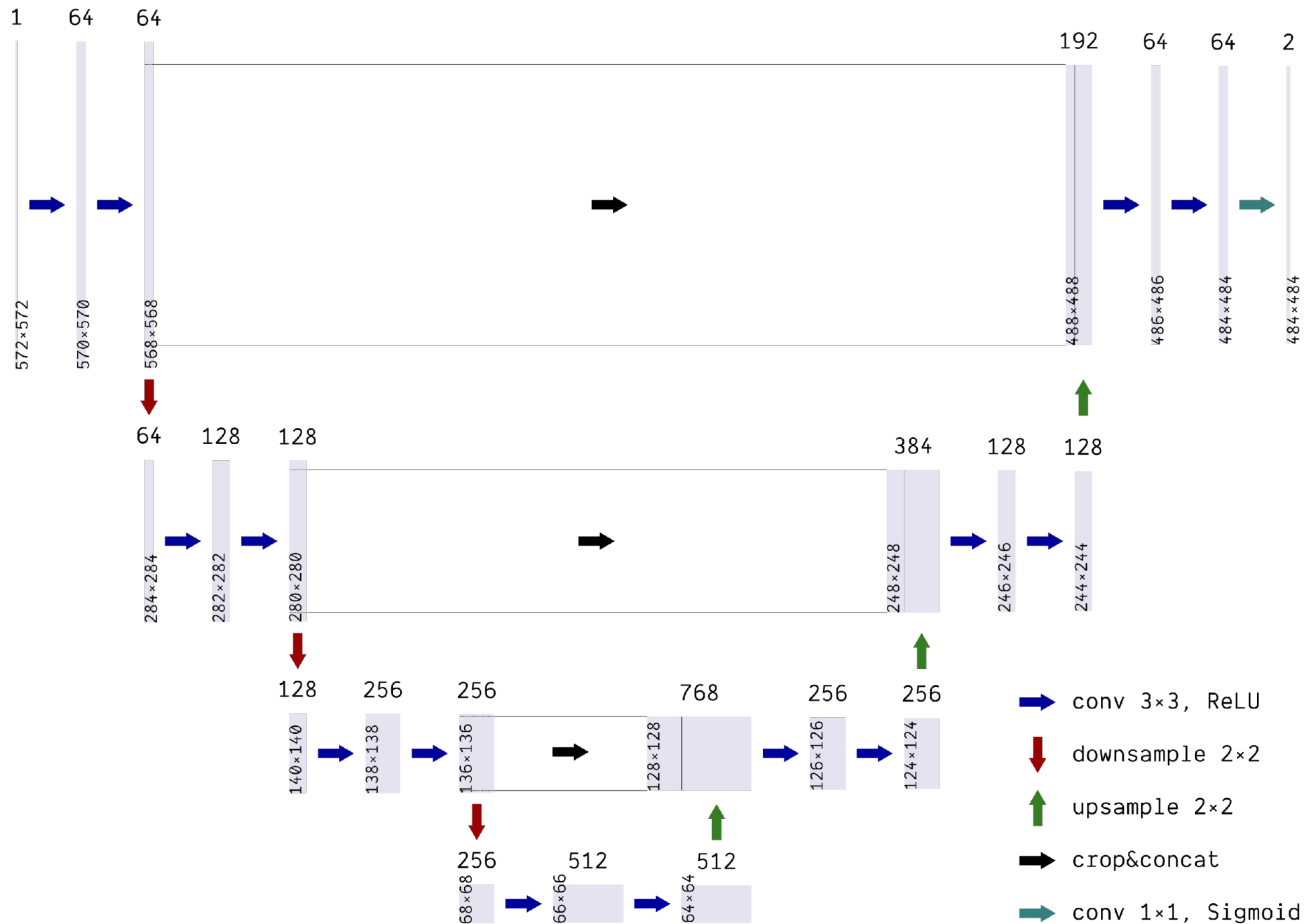


# receptive field

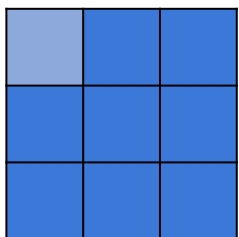
*set of input units that can influence the value of a specific output unit.*

**convolution x 2 + max pooling + convolution x2  
+ max pooling + convolution x 2**



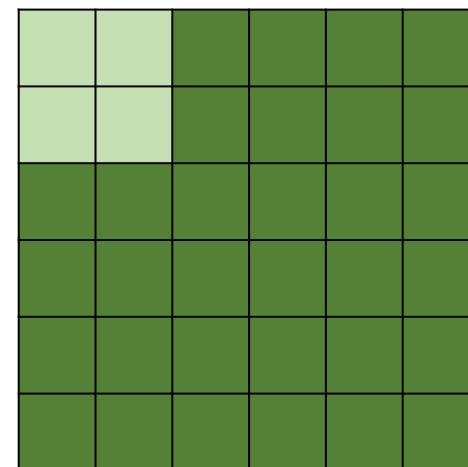


# upsampling



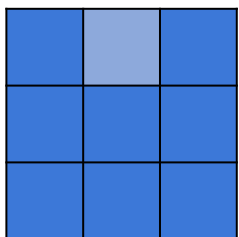
some layer

\* nearest-neighbor  
/ bilinear



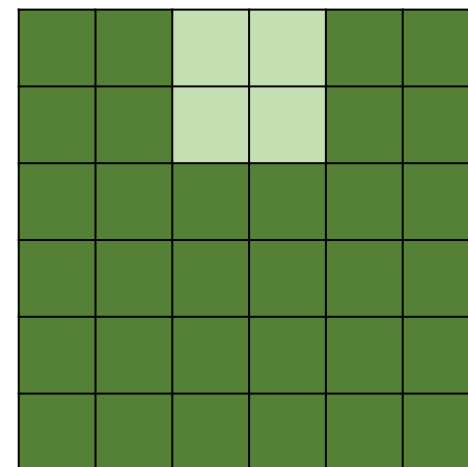
next layer

# upsampling



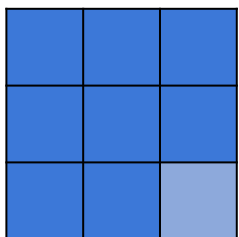
some layer

\* nearest-neighbor  
/ bilinear



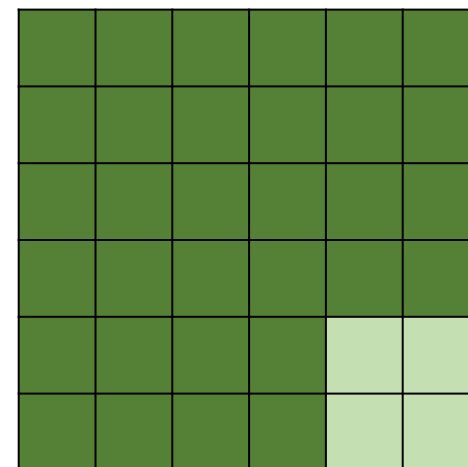
next layer

# upsampling



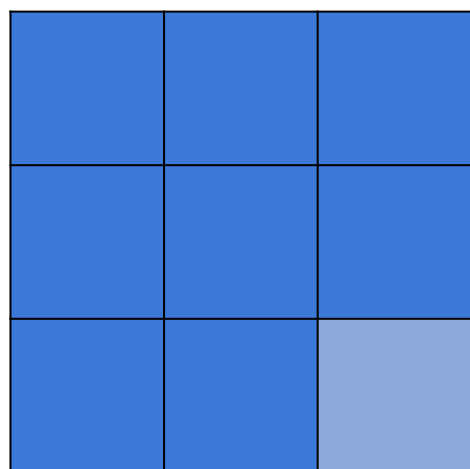
some layer

\* nearest-neighbor  
/ bilinear



next layer

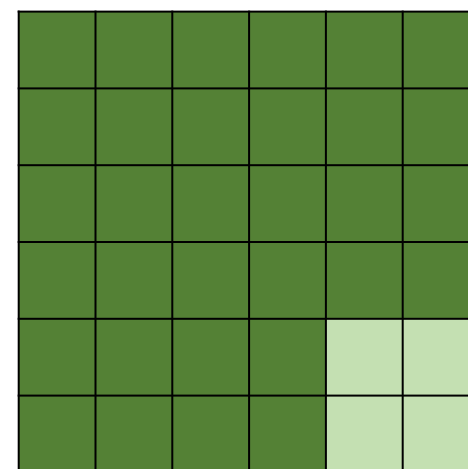
# upsampling



some layer

\*

nearest-neighbor  
/ bilinear

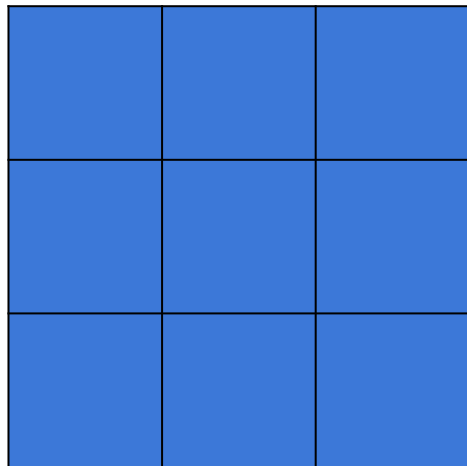


next layer

# receptive field

*set of input units that can influence the value of a specific output unit.*

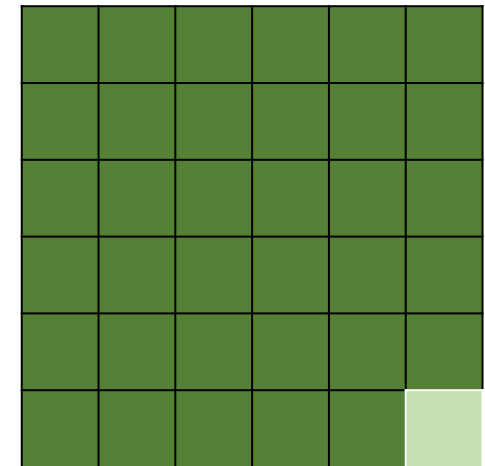
## nearest neighbor upsampling



some layer

\*

nearest-neighbor

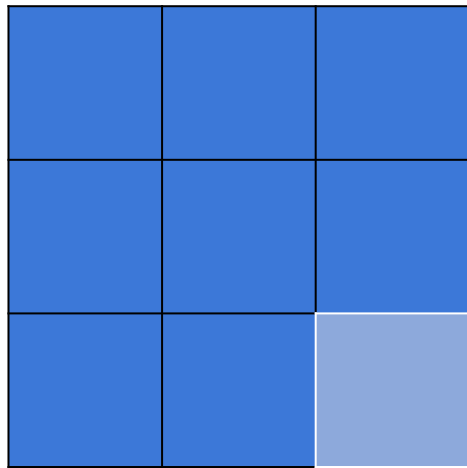


next layer

# receptive field

*set of input units that can influence the value of a specific output unit.*

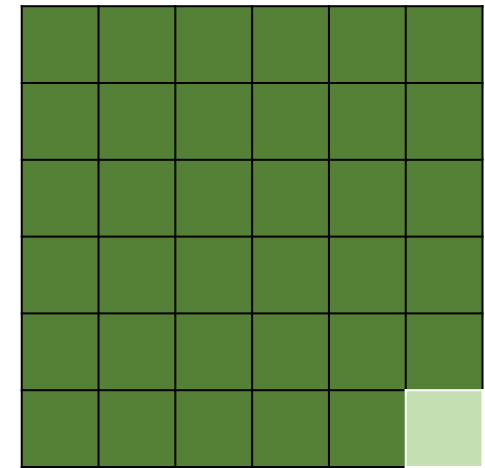
## nearest neighbor upsampling



some layer

\*

nearest-neighbor



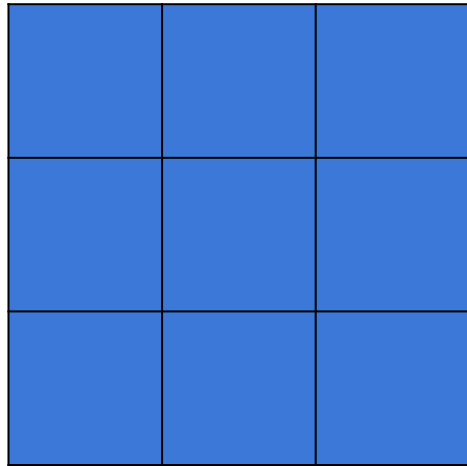
next layer



# receptive field

*set of input units that can influence the value of a specific output unit.*

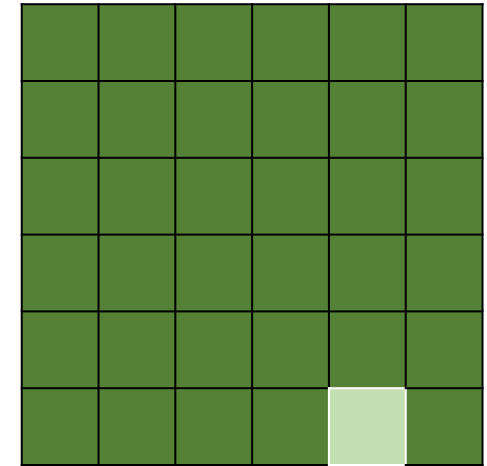
## nearest neighbor upsampling



some layer

\*

nearest-neighbor

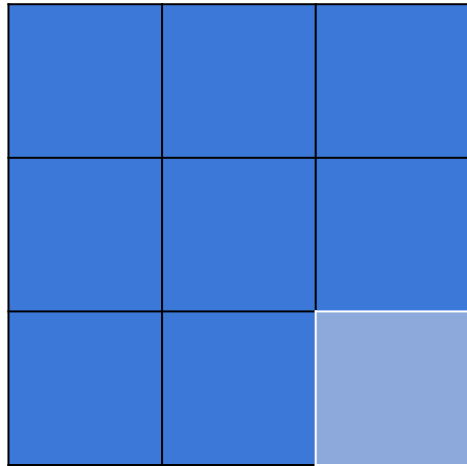


next layer

# receptive field

*set of input units that can influence the value of a specific output unit.*

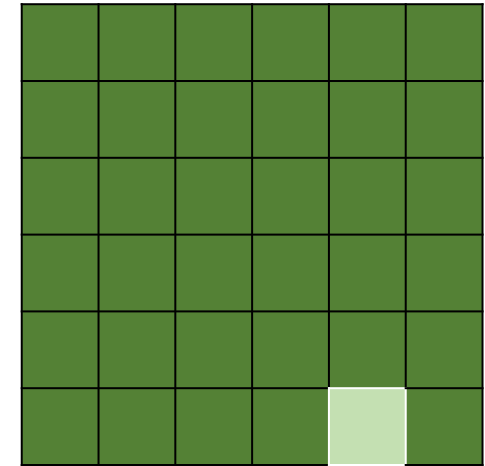
## nearest neighbor upsampling



some layer

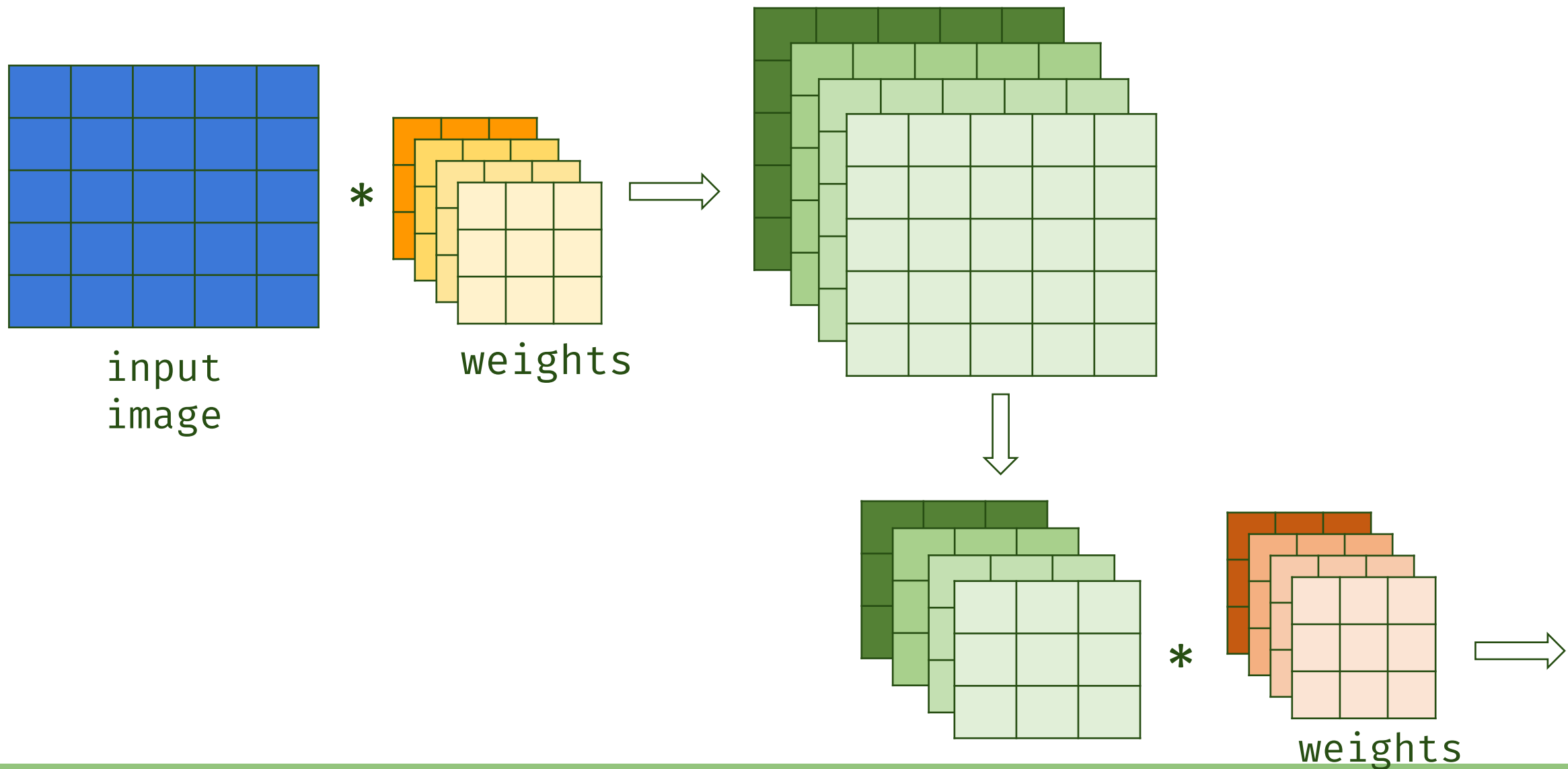
\*

nearest-neighbor

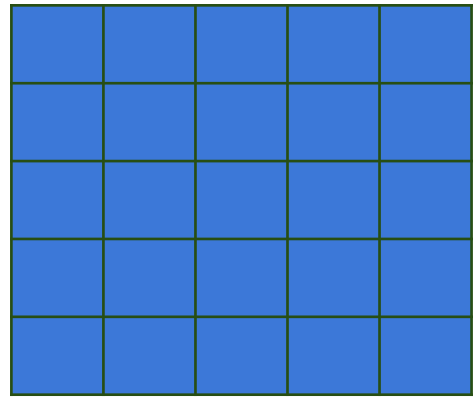


next layer

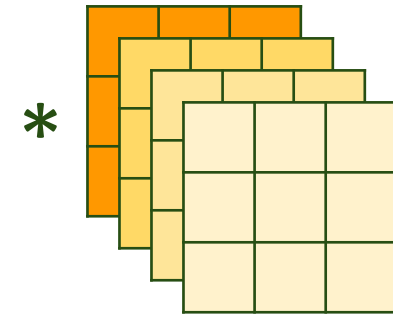
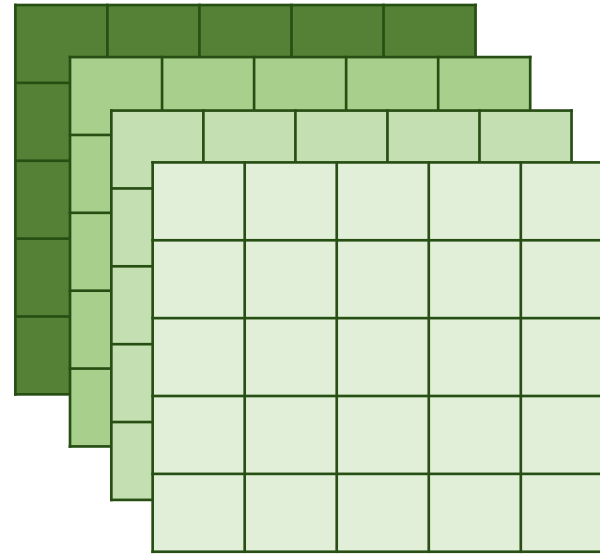
# building an encoder



# building a decoder

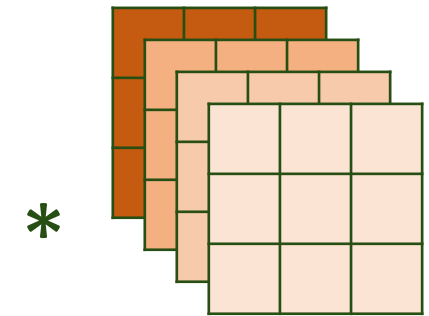
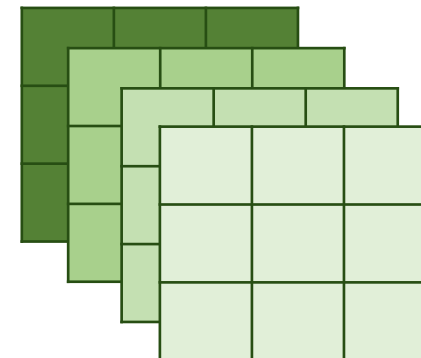


output  
image



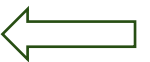
\*

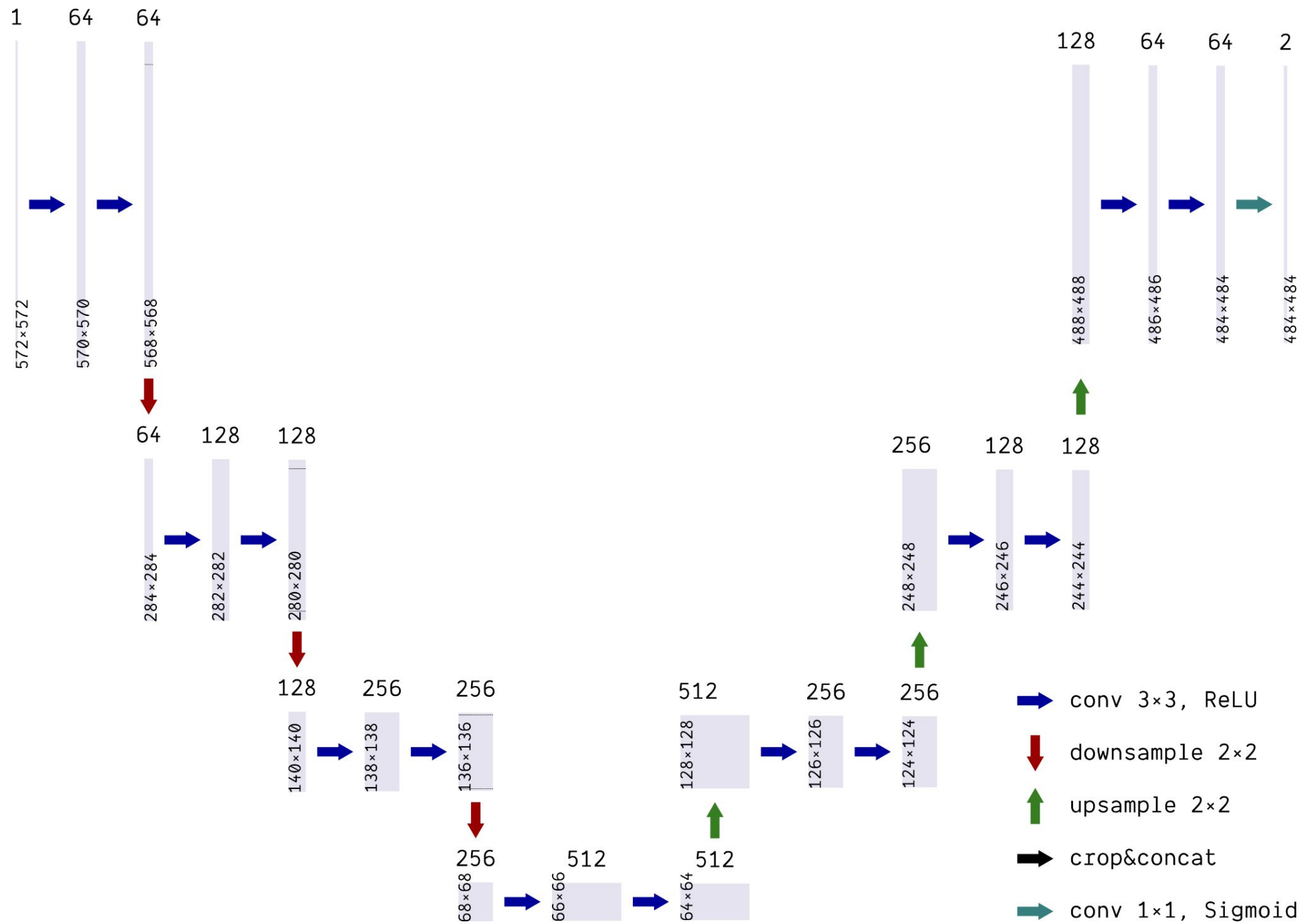
weights

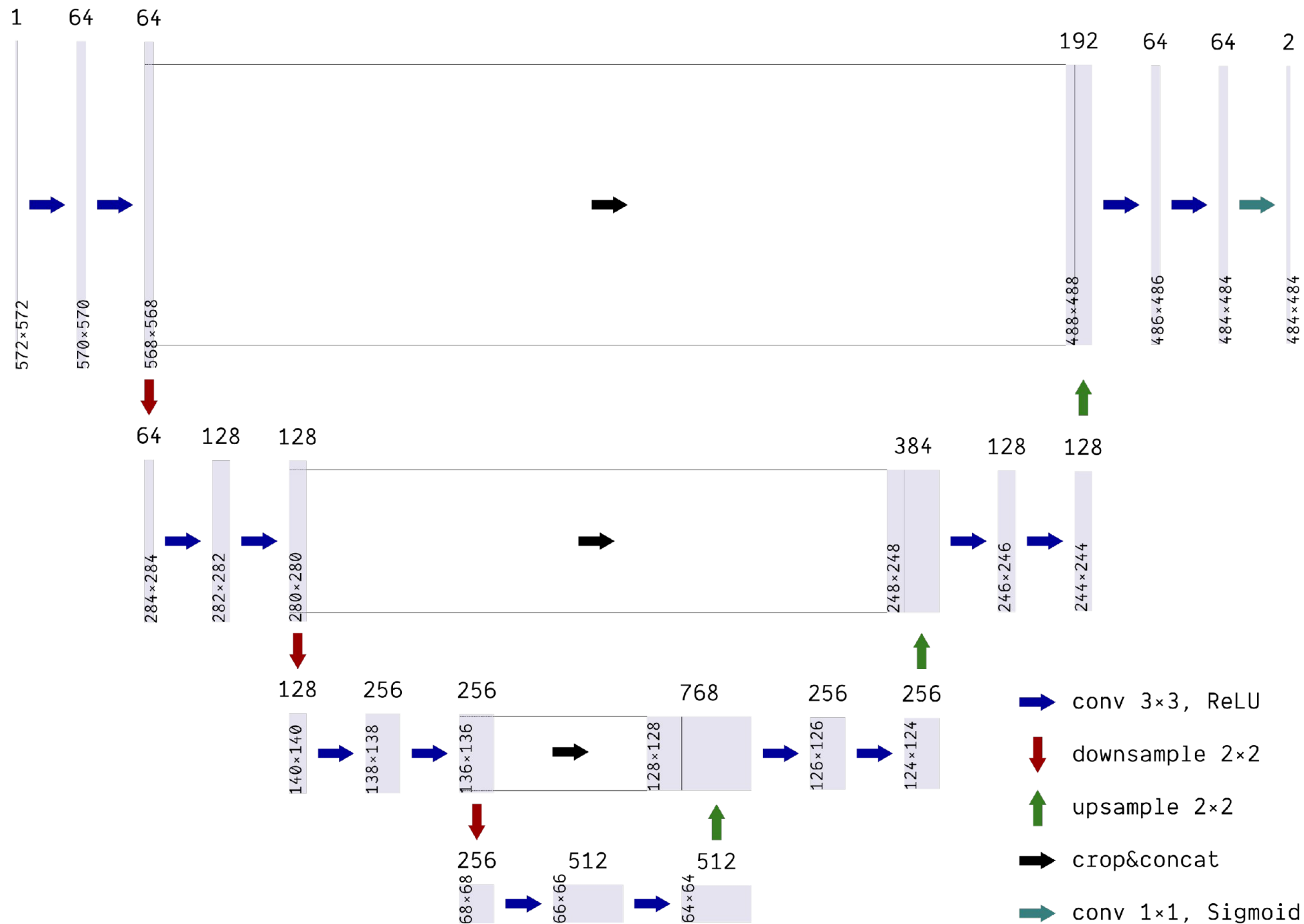


\*

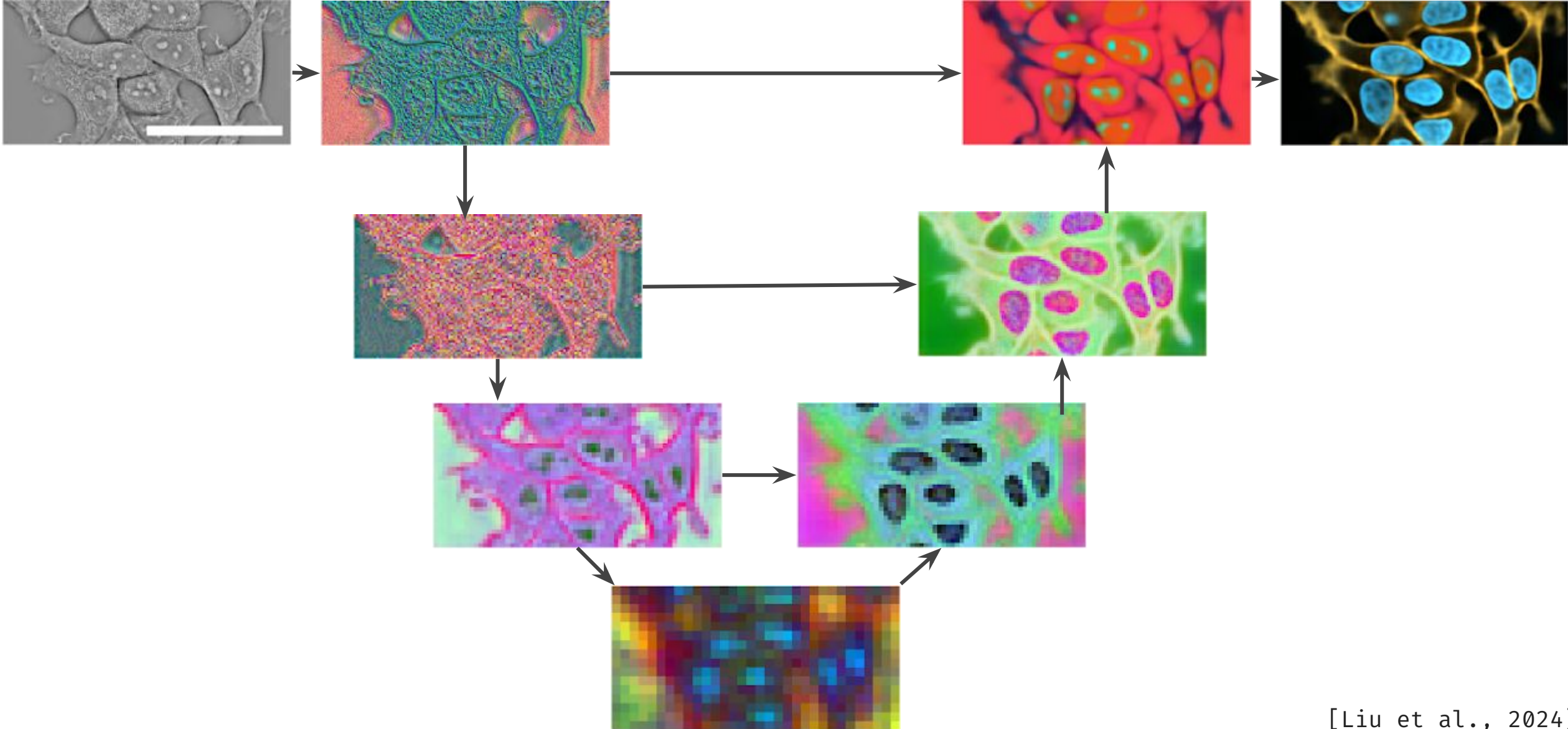
weights





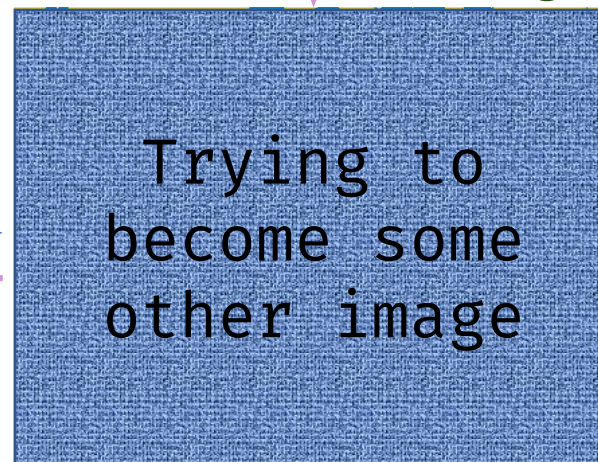
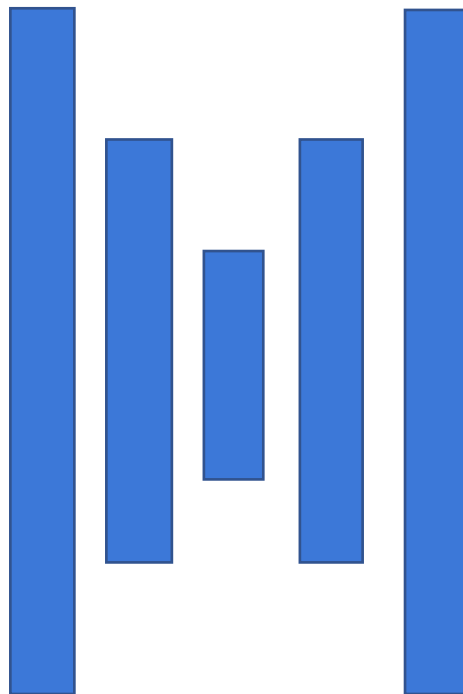
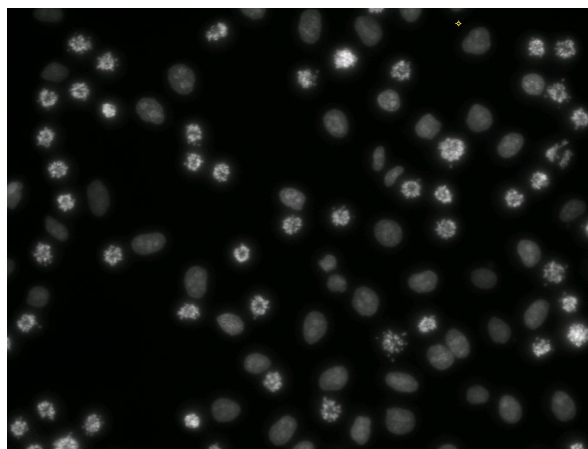


# multi-scale representation



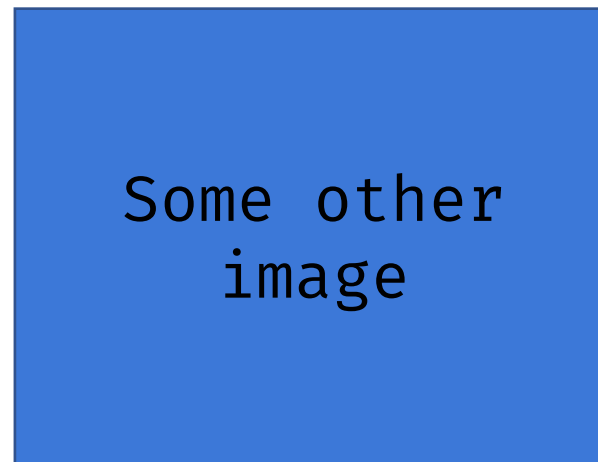
# training

→ Forward pass  
← Backpropagation



Current output:  
transformed image

Groundtruth



Loss for  
training



Metric for  
validation





# questions you may have (asked)

- How big does my receptive field need to be?
- What about image/patch size?
- Bigger images or larger batches?

**Go build your own U-Net!**